Why can we train deep neural networks?

$$\text{depth} \rightarrow \text{stability} \rightarrow \text{optimization} \rightarrow \text{regularization} \rightarrow \text{residual learning}$$

Roadmap

1. Signal propagation — how scale moves through depth
2. Initialization — choosing the starting scale
3. Optimization — how gradients become steps
4. Normalization and regularization — stable scales and generalization
5. Residual learning and diagnostics — direct paths and debugging

Each part answers one reason why deep networks are hard to train, and one tool that fixes it.

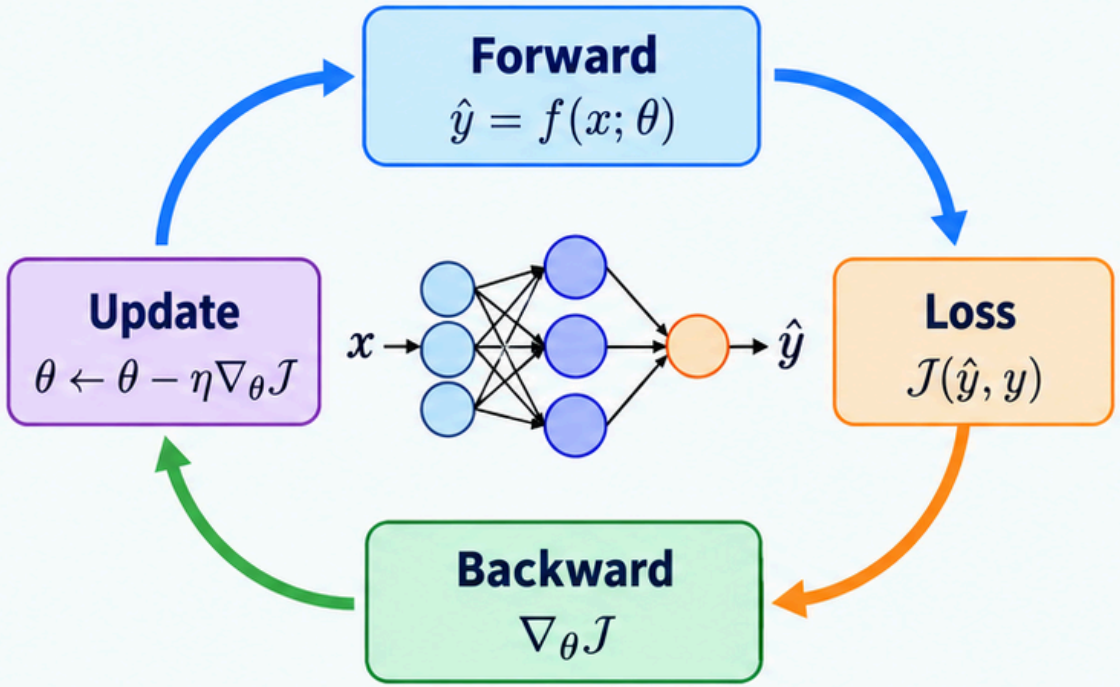
From learning to deep training

Session 01 established the learning loop:

$$\text{forward} \rightarrow \text{loss} \rightarrow \text{backward} \rightarrow \text{update}$$

Now we ask what happens when this loop is applied to **deep compositions**.

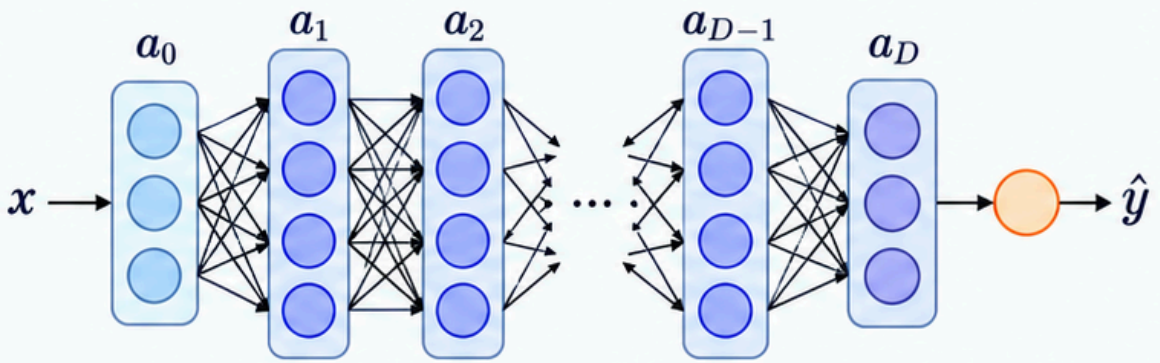
Single learning loop (shallow model)



PyTorch

```
optimizer.zero_grad()
y_hat = model(x)
J = loss_fn(y_hat, y)
J.backward()
optimizer.step()
```

Same loop, deep trainable system



$$z_l = W_l a_{l-1} + b_l, \quad a_l = \varphi(z_l) \quad (l = 1, \ldots, D)$$

1. Signal propagation (through depth)

$$z_l = W_l a_{l-1} + b_l$$
$$a_l = \varphi(z_l)$$

activations $a_1, a_2, \ldots, a_D$

2. Optimization (gradient-based)

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{J}$$

(e.g., SGD, Adam)

3. Regularization & stability

- Initialization
- Normalization
- Optimizer
- Regularization
- Residual / skip paths

The loop stays the same; depth changes signal propagation, optimization, and stability.

The loop remains the same

The PyTorch loop does not change:

```
optimizer.zero_grad()
y_hat = model(x)
loss = criterion(y_hat, y)
loss.backward()
optimizer.step()
```

What changes is the behavior of the model as depth increases.

A deep network is a composition

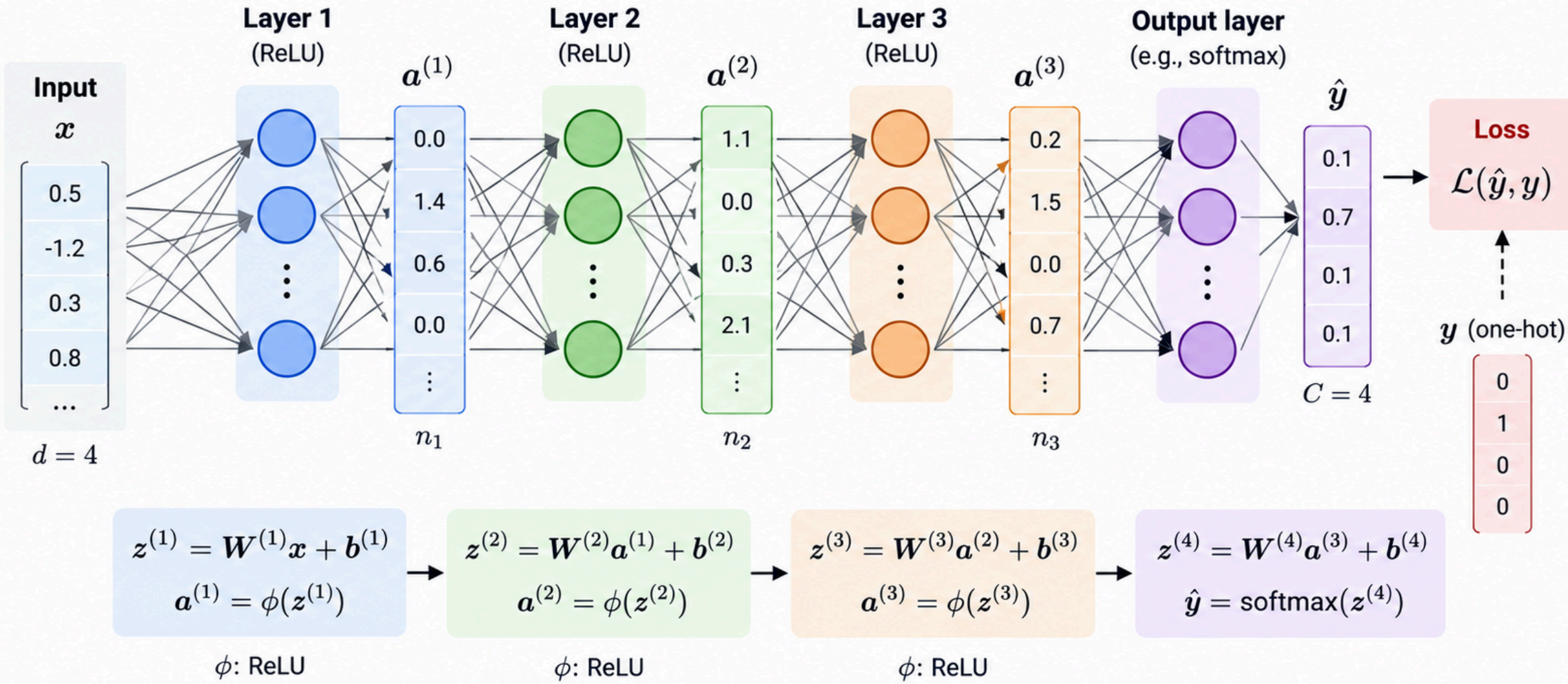
For layer $l = 1, \ldots, D$, with $a_0 = x$:

$$z_l = W_l a_{l-1} + b_l, \qquad a_l = \phi(z_l)$$

Therefore, for a network of depth D:

$$f_\theta(x) = f_D \circ f_{D-1} \circ \cdots \circ f_1(x)$$

Same notation as S01: ϕ is a generic activation, L is the loss.


Input
x
0.5
-1.2
0.3
0.8
...
$d = 4$
Layer 1
(ReLU)
$a^{(1)}$
0.0
1.4
0.6
0.0
n_1
Layer 2
(ReLU)
$a^{(2)}$
1.1
0.0
0.3
2.1
n_2
Layer 3
(ReLU)
$a^{(3)}$
0.2
1.5
0.0
0.7
n_3
Output layer
(e.g., softmax)
$\hat{y}$
0.1
0.7
0.1
0.1
$C = 4$
Loss
$\mathcal{L}(\hat{y}, y)$
y (one-hot)
0
1
0
0
$z^{(1)} = W^{(1)}x + b^{(1)}$
$a^{(1)} = \phi(z^{(1)})$
ϕ: ReLU
$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$
$a^{(2)} = \phi(z^{(2)})$
ϕ: ReLU
$z^{(3)} = W^{(3)}a^{(2)} + b^{(3)}$
$a^{(3)} = \phi(z^{(3)})$
ϕ: ReLU
$z^{(4)} = W^{(4)}a^{(3)} + b^{(4)}$
$\hat{y} = \text{softmax}(z^{(4)})$

Depth changes the training problem

Each layer transforms:

- the representation;
- the scale of activations;
- the direction and magnitude of gradients;
- the geometry seen by the optimizer.

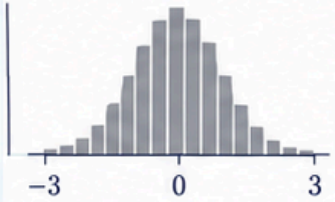
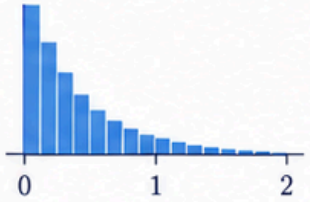
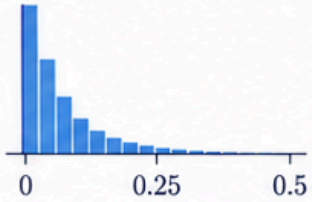
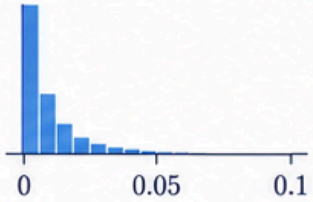
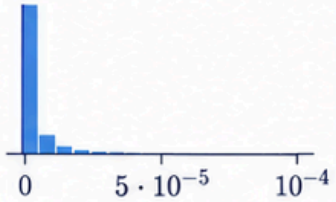
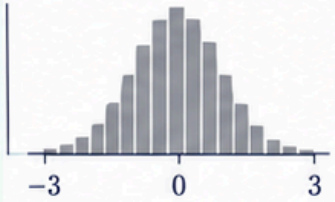
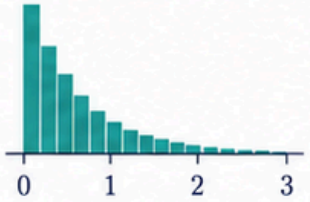
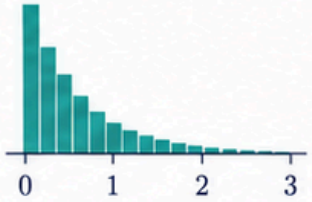
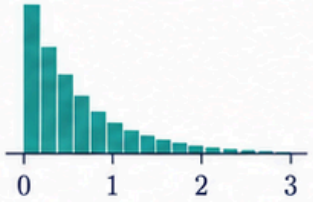
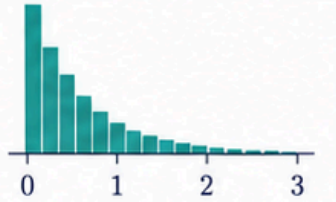
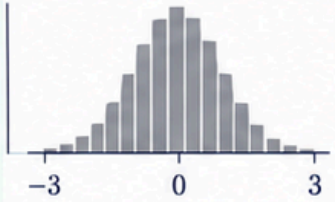
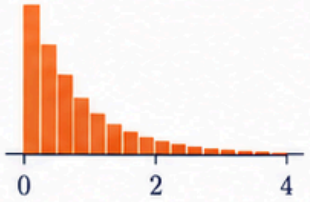
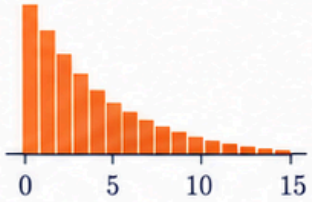
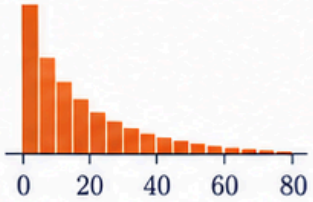
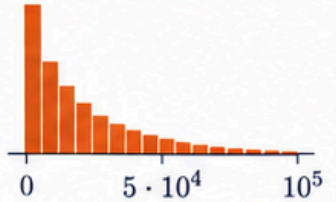
The issue is not only **expressiveness**.
The issue is also **trainability**.

Signal propagation

A useful deep network should keep information in a usable numerical range.

$$x \to a_1 \to a_2 \to \cdots \to a_D$$

$$a_l = \mathrm{ReLU}(W_l a_{l-1}),$$
$$(W_l)_{ij} \sim \mathcal{N}(0, c \cdot 2/n_{\text{in}}).$$

	input x	layer 1	layer 5	layer 10	layer 30	
$c = 0.5$ (too small)	 $\mathrm{std}(x) = 1.0$	 $\mathrm{std}(a_l) = 0.58$	 $\mathrm{std}(a_l) = 0.15$	 $\mathrm{std}(a_l) = 0.026$	 $\mathrm{std}(a_l) = 2.5 \cdot 10^{-5}$	→ signal vanishes
$c = 1$ (He)	 $\mathrm{std}(x) = 1.0$	 $\mathrm{std}(a_l) = 0.82$	 $\mathrm{std}(a_l) = 0.82$	 $\mathrm{std}(a_l) = 0.82$	 $\mathrm{std}(a_l) = 0.82$	→ scale preserved
$c = 2$ (too large)	 $\mathrm{std}(x) = 1.0$	 $\mathrm{std}(a_l) = 1.2$	 $\mathrm{std}(a_l) = 4.7$	 $\mathrm{std}(a_l) = 26$	 $\mathrm{std}(a_l) = 2.7 \cdot 10^4$	→ signal explodes

Each layer multiplies the activation scale by about $\sqrt{c}$; over 30 layers that factor is raised to the 30th power.

Prediction before experiment

Suppose we build a 50-layer MLP with random weights.

What do you expect to happen to:

$$\mathrm{mean}(a_l), \qquad \mathrm{std}(a_l)$$

as l increases?

Activations may vanish or explode

If layer transformations systematically shrink the signal:

$$\mathrm{std}(a_l) \to 0$$

If they systematically amplify it:

$$\mathrm{std}(a_l) \to \infty$$

Both cases make optimization difficult.

Gradients also propagate through depth

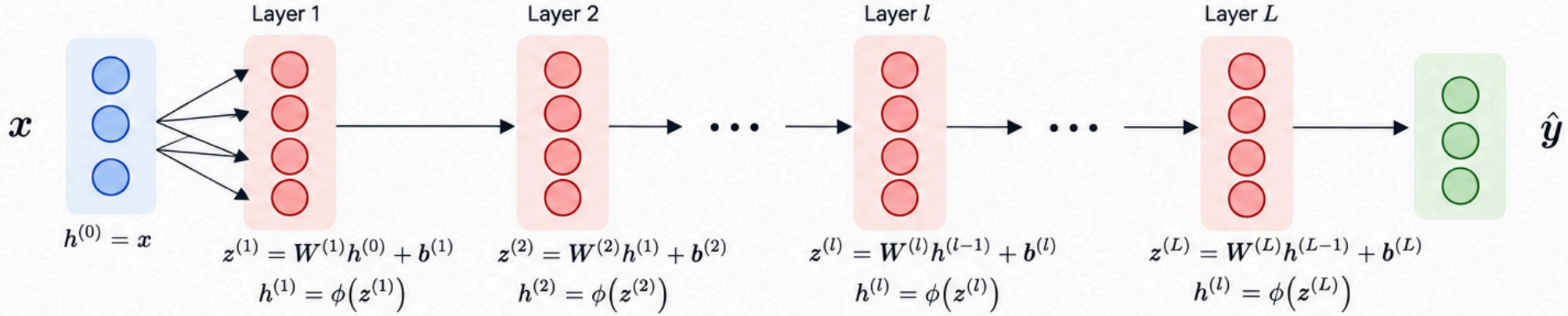
Backpropagation traverses the graph in reverse. With $\delta_{a_l} = \partial L / \partial a_l$ as in S01:

$$\delta_{a_l} = \left(\frac{\partial a_{l+1}}{\partial a_l} \right)^{\top} \delta_{a_{l+1}} = W_{l+1}^{\top} \left(\delta_{a_{l+1}} \odot \phi'(z_{l+1}) \right)$$

Across D layers, the chain rule becomes a long **product** of local factors.

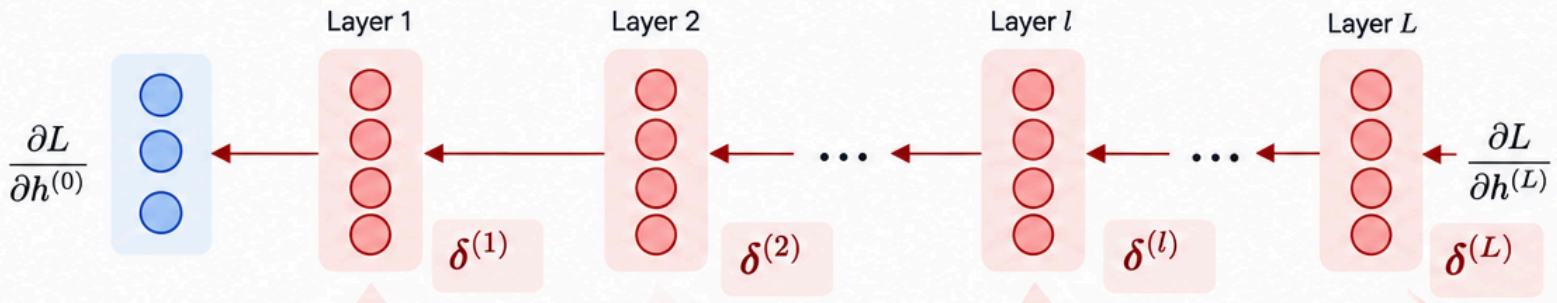
Forward pass (activations)

Each layer transforms the input representation.

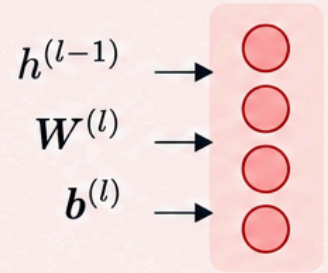


Backward pass (gradients)

The gradient flows through many transformations using the chain rule.



Local backward computation (layer l)



$$\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}} = \left(W^{(l+1)}\right)^{\top} \delta^{(l+1)} \odot \phi'\left(z^{(l)}\right)$$

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \left(h^{(l-1)}\right)^{\top} \qquad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$$

Gradient across many layers

$$\frac{\partial L}{\partial h^{(0)}} = \underbrace{\left(\prod_{l=1}^{L} J^{(l)}\right)}_{\text{Product of Jacobians}} \frac{\partial L}{\partial h^{(L)}}$$

$$J^{(l)} = \operatorname{diag}\left(\phi'\left(z^{(l)}\right)\right) W^{(l)}$$

Repeated multiplication can make gradients very small (vanish) or very large (explode), depending on the scales of $J^{(l)}$.

Vanishing and exploding gradients

The optimizer updates parameters using gradients.

If gradients vanish:

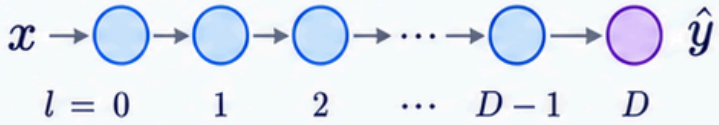
$$\|\nabla_{W_l} J\| \approx 0$$

early layers barely learn.

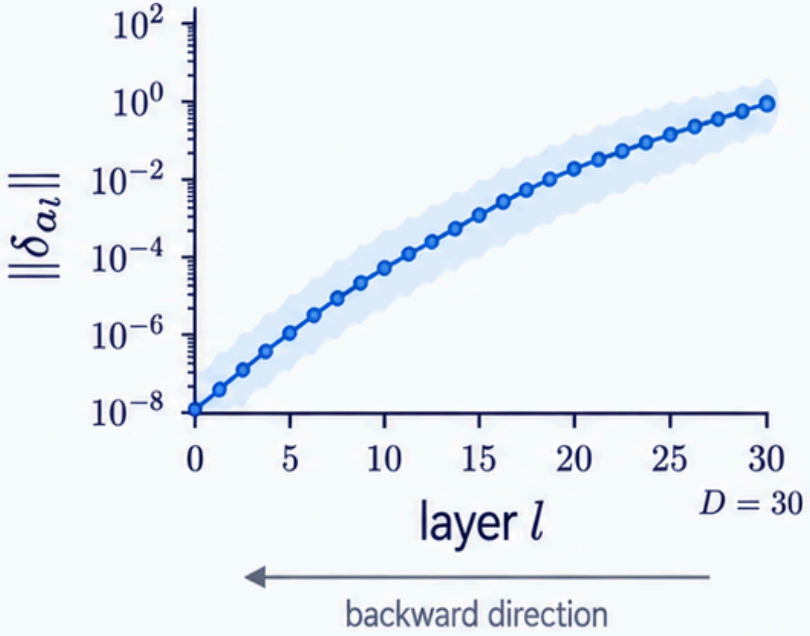
If gradients explode, updates become unstable.

Vanishing gradients

backward signal (backpropagation)

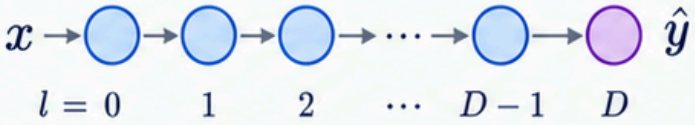


$\|\partial a_l / \partial a_{l-1}\| < 1$ on average

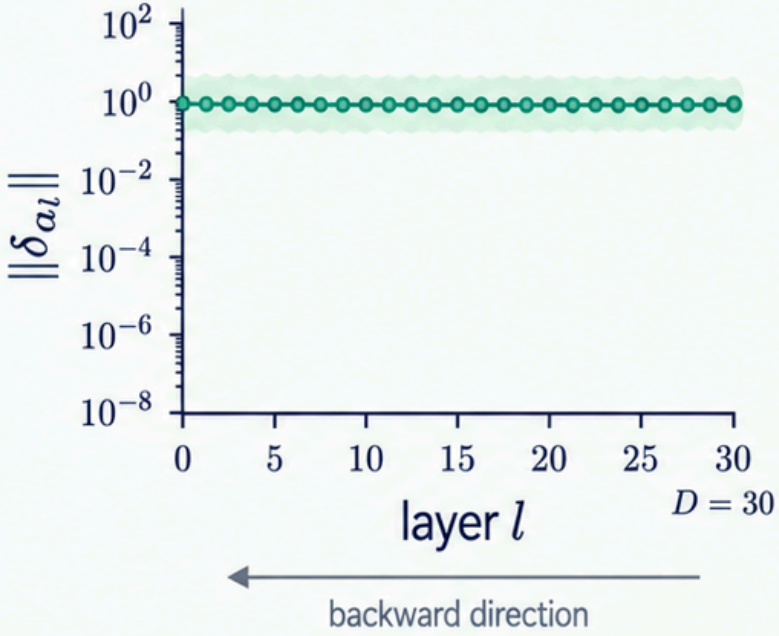


Stable gradients

backward signal (backpropagation)

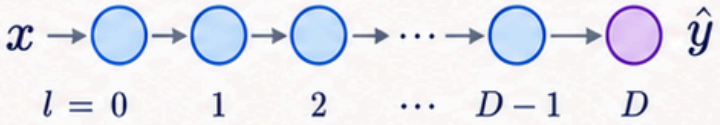


$\|\partial a_l / \partial a_{l-1}\| \approx 1$ on average

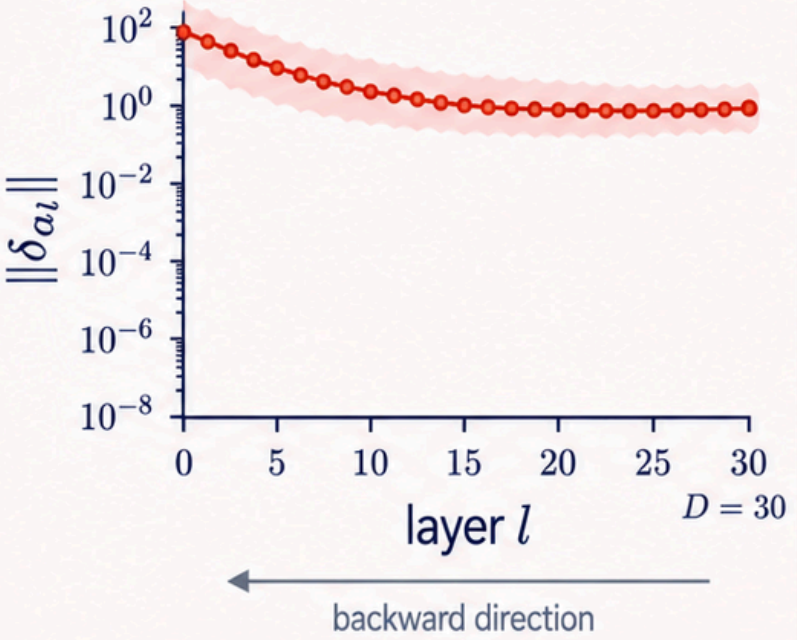


Exploding gradients

backward signal (backpropagation)



$\|\partial a_l / \partial a_{l-1}\| > 1$ on average



During backpropagation, repeated multiplication by layer Jacobians can shrink, preserve, or amplify the backward signal.

Activation functions shape gradient flow

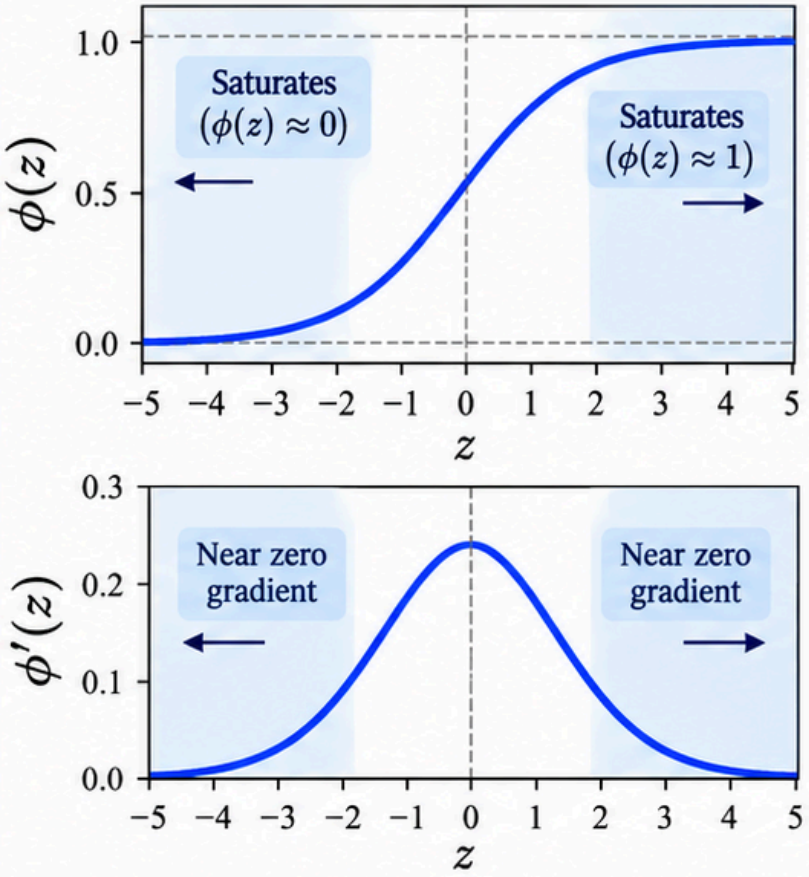
Saturating activations can compress gradients:

$$\sigma'(z) \leq 0.25, \qquad \tanh'(z) \leq 1$$

ReLU-like activations can preserve stronger gradient paths, but they also introduce zero-gradient regions.

Sigmoid

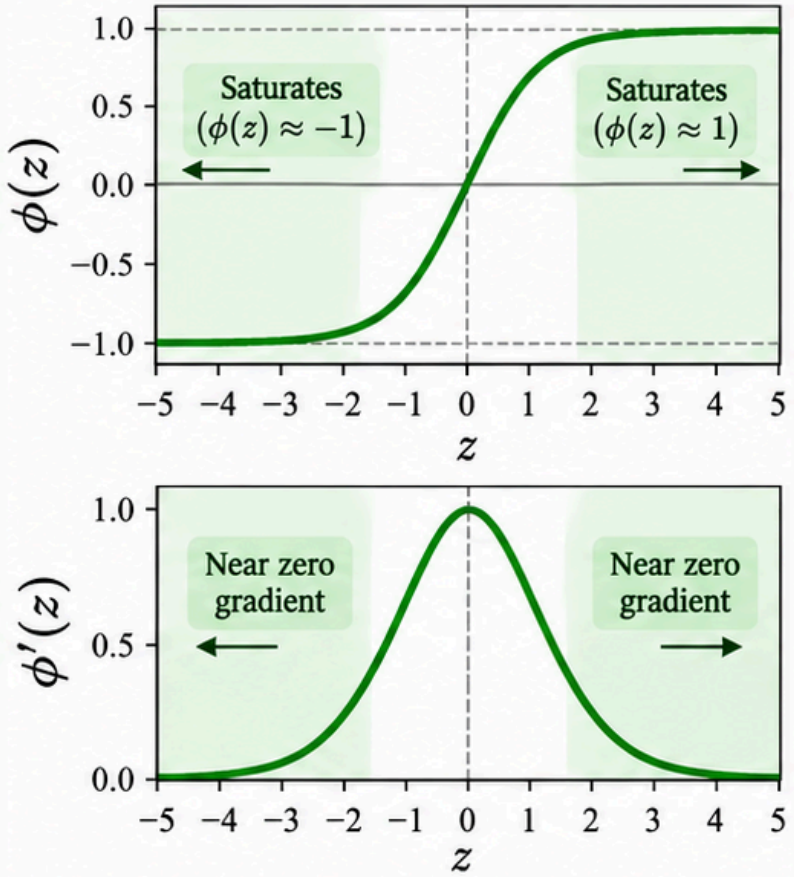
$$\phi(z) = \frac{1}{1+e^{-z}}$$



Not zero-centered; strong saturation can shrink gradients.

Tanh

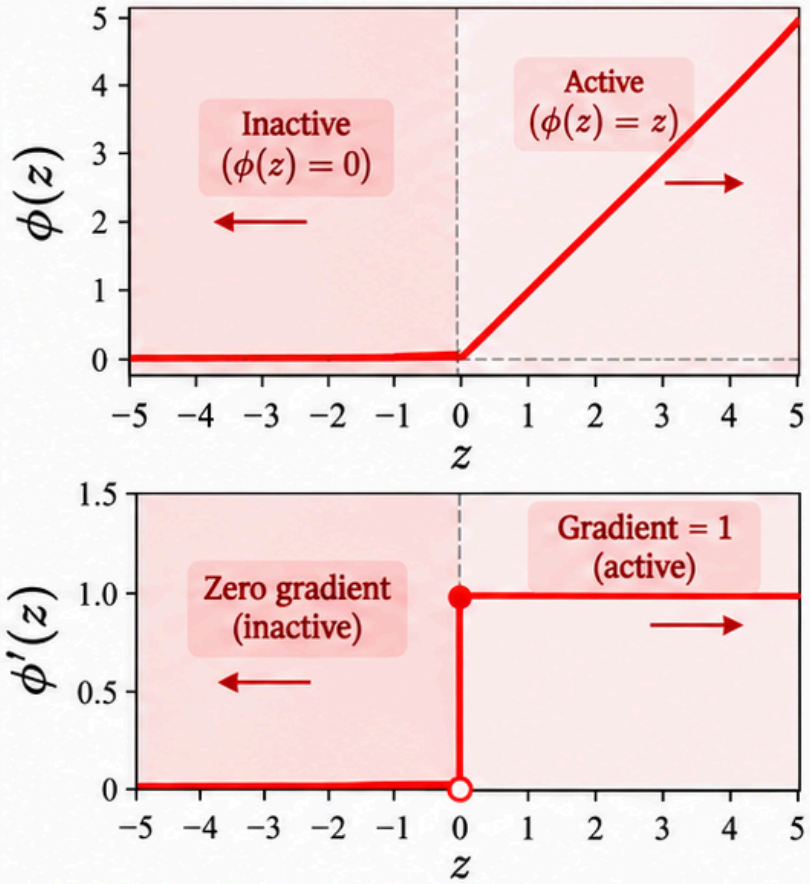
$$\phi(z) = \tanh(z)$$



Zero-centered output, but still saturates for large $|z|$.

ReLU

$$\phi(z) = \max(0,\ z)$$



No positive-side saturation; gradients can die for inactive units.

Activation choice affects gradient flow through depth.

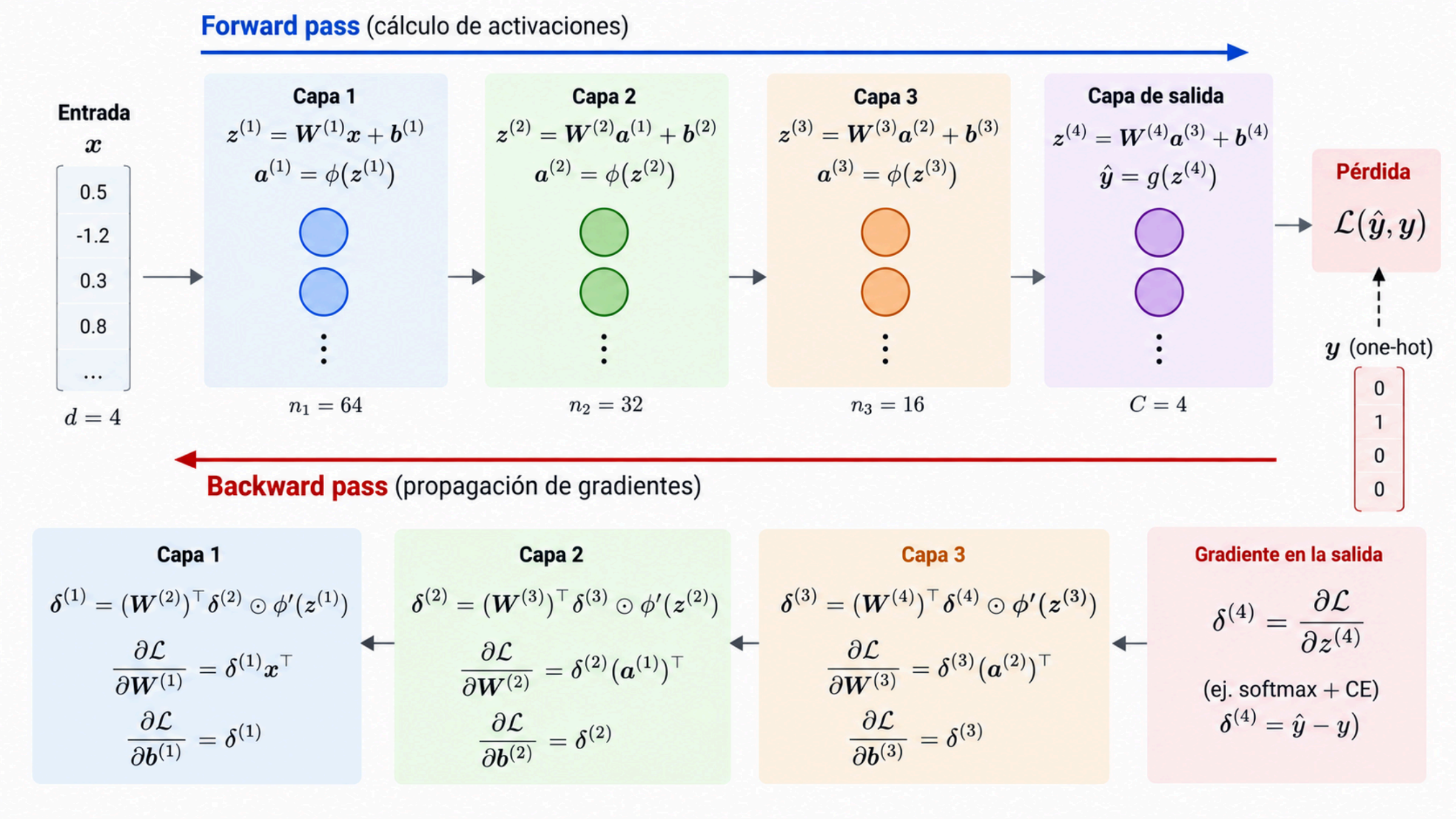
Dead ReLUs and ReLU variants

For ReLU, $\phi'(z) = 0$ when $z < 0$. A unit with $z < 0$ for **every** input outputs 0 and receives no gradient: it is **dead** and never recovers.

Typical causes: a large update that pushes its bias far below zero, or a learning rate that is too large.

$$\text{Leaky ReLU: } \phi(z) = \max(\alpha z,\, z), \;\; \alpha \approx 0.01 \qquad \text{GELU: } \phi(z) = z\,\Phi(z)$$

Φ: standard normal CDF. GELU is the default activation in Transformers (S06).


Forward pass (cálculo de activaciones)
Entrada
x
0.5
-1.2
0.3
0.8
...
$d = 4$
Capa 1
$z^{(1)} = W^{(1)}x + b^{(1)}$
$a^{(1)} = \phi(z^{(1)})$
$n_1 = 64$
Capa 2
$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$
$a^{(2)} = \phi(z^{(2)})$
$n_2 = 32$
Capa 3
$z^{(3)} = W^{(3)}a^{(2)} + b^{(3)}$
$a^{(3)} = \phi(z^{(3)})$
$n_3 = 16$
Capa de salida
$z^{(4)} = W^{(4)}a^{(3)} + b^{(4)}$
$\hat{y} = g(z^{(4)})$
$C = 4$
Pérdida
$\mathcal{L}(\hat{y}, y)$
y (one-hot)
0
1
0
0
Backward pass (propagación de gradientes)
Gradiente en la salida
$\delta^{(4)} = \frac{\partial \mathcal{L}}{\partial z^{(4)}}$
(ej. softmax + CE)
$\delta^{(4)} = \hat{y} - y$
Capa 3
$\delta^{(3)} = (W^{(4)})^\top \delta^{(4)} \odot \phi'(z^{(3)})$
$\frac{\partial \mathcal{L}}{\partial W^{(3)}} = \delta^{(3)}(a^{(2)})^\top$
$\frac{\partial \mathcal{L}}{\partial b^{(3)}} = \delta^{(3)}$
Capa 2
$\delta^{(2)} = (W^{(3)})^\top \delta^{(3)} \odot \phi'(z^{(2)})$
$\frac{\partial \mathcal{L}}{\partial W^{(2)}} = \delta^{(2)}(a^{(1)})^\top$
$\frac{\partial \mathcal{L}}{\partial b^{(2)}} = \delta^{(2)}$
Capa 1
$\delta^{(1)} = (W^{(2)})^\top \delta^{(2)} \odot \phi'(z^{(1)})$
$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \delta^{(1)}x^\top$
$\frac{\partial \mathcal{L}}{\partial b^{(1)}} = \delta^{(1)}$

Part 2 · Initialization

1. Signal propagation — how scale moves through depth
2. **Initialization** — choosing the starting scale
3. Optimization — how gradients become steps
4. Normalization and regularization — stable scales and generalization
5. Residual learning and diagnostics — direct paths and debugging

> Part 1: each layer multiplies the forward and the backward signal by a similar factor; depth turns that factor into a power.

Initialization is not a detail

Training starts before the first update.

The initial distribution of weights determines the first forward pass and the first backward pass.

$$(W_l)_{ij} \sim \text{some distribution}$$

The question is: **which distribution keeps signals stable?**

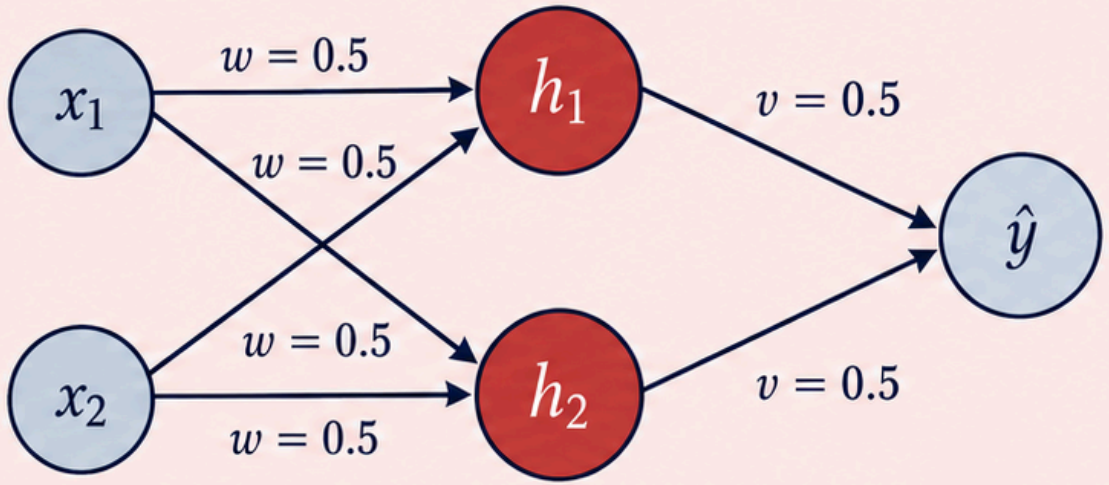
Why not initialize everything equally?

If two hidden units start with the same parameters, they compute the same value.

They also receive the same gradient.

So they remain identical.

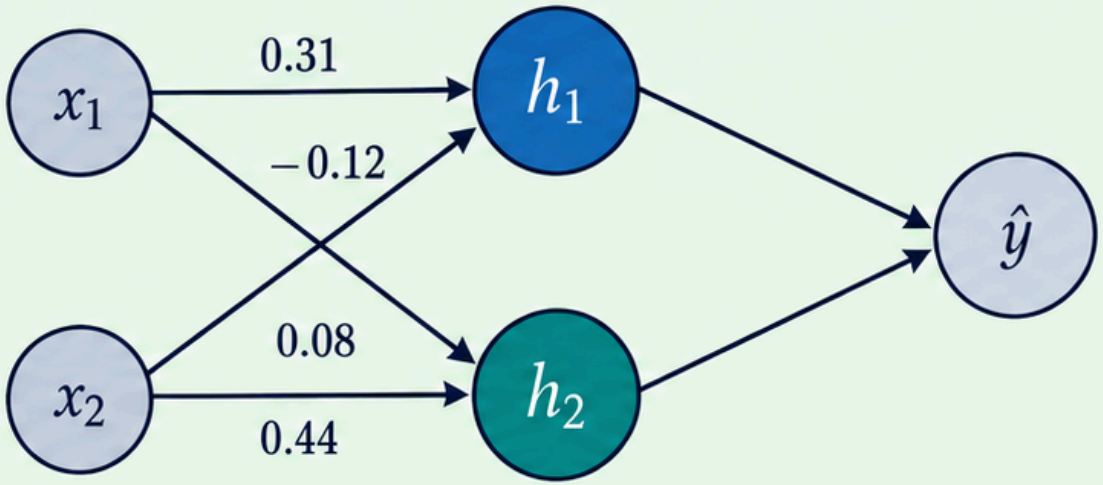
identical initialization



=	forward: $a_1 = a_2$
=	backward: $\partial J / \partial W_{1,:} = \partial J / \partial W_{2,:}$
=	after the update: still $W_{1,:} = W_{2,:}$

every step: the two units stay copies of each other

random initialization



≠	forward: $a_1 \neq a_2$
≠	backward: different gradients
≠	the units learn different features

Identical units compute the same value and receive the same gradient, so they never diverge. Random initialization breaks the symmetry.

Preserve variance across layers

A useful initialization should avoid systematic shrinkage or growth in the forward pass:

$$\mathrm{Var}(a_l) \approx \mathrm{Var}(a_{l-1})$$

and ideally also in the backward pass:

$$\mathrm{Var}(\delta_{a_l}) \approx \mathrm{Var}(\delta_{a_{l+1}})$$

Where the initialization scale comes from

For one unit, with independent zero-mean weights:

$$z_{l,i} = \sum_{j=1}^{n_{\text{in}}} (W_l)_{ij}\, a_{l-1,j} \quad \Rightarrow \quad \operatorname{Var}(z_l) = n_{\text{in}}\, \operatorname{Var}(W_l)\, \mathbb{E}\left[a_{l-1}^2\right]$$

- tanh near 0 behaves like the identity: $\mathbb{E}[a^2] \approx \operatorname{Var}(z)$, so keep $n_{\text{in}} \operatorname{Var}(W) = 1$
- ReLU zeroes half of a symmetric input: $\mathbb{E}[a^2] = \frac{1}{2}\operatorname{Var}(z)$, so keep $n_{\text{in}} \operatorname{Var}(W) = 2$

The backward pass gives the same condition with n_{out}. Each layer multiplies the variance by a constant; over D layers that constant is raised to the power D.

Xavier / Glorot initialization

For activations such as $\tanh$, Xavier initialization chooses a scale based on layer fan-in and fan-out:

$$(W_l)_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right)$$

This is $\text{Var}(W) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$: the average of the forward and backward conditions.

$\mathcal{U}(-r, r)$ has variance $r^2/3$.

He / Kaiming initialization

For ReLU, the condition $n_{\text{in}}\,\text{Var}(W) = 2$ gives:

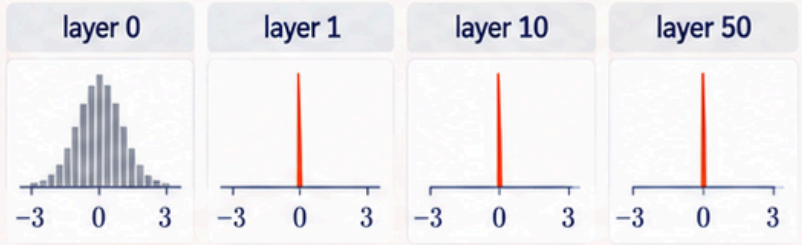
$$(W_l)_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

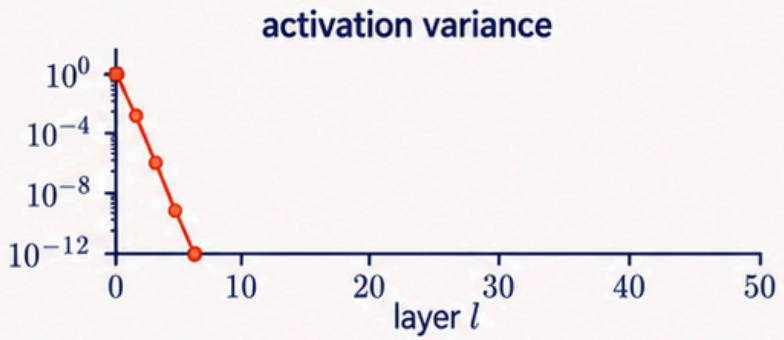
Initialization and activation must be considered together.

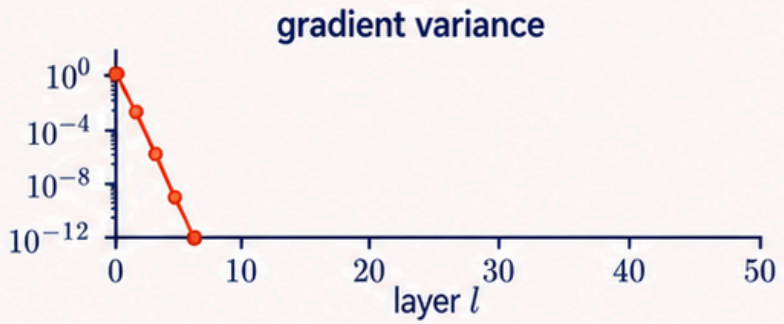
Small initialization · ReLU

$W \sim \mathcal{N}(0, 0.01^2)$

$n_{\text{in}} = 256$: variance × 0.013 per layer

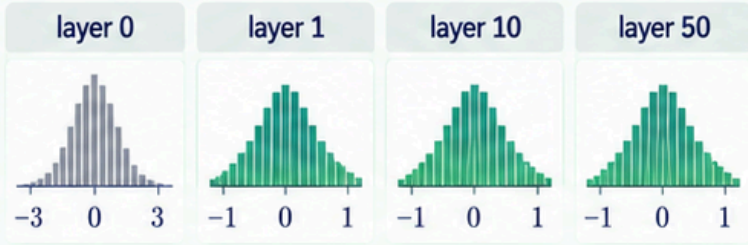


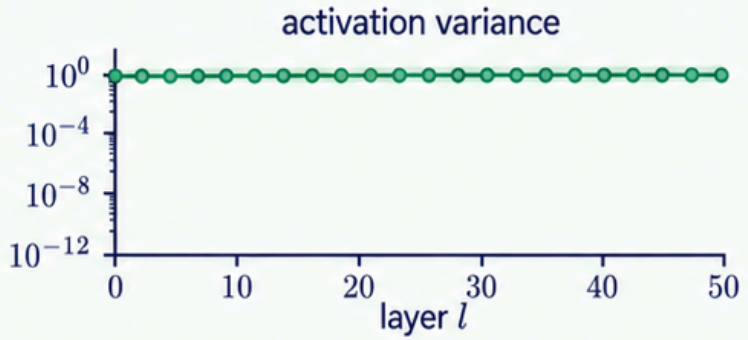


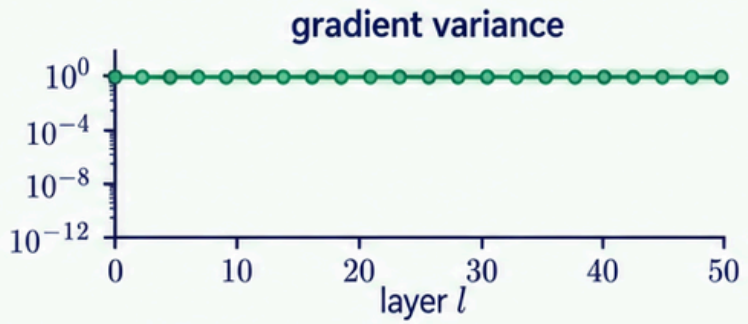


Xavier (Glorot) · tanh

$W \sim \mathcal{N}(0, 2/(n_{\text{in}} + n_{\text{out}}))$

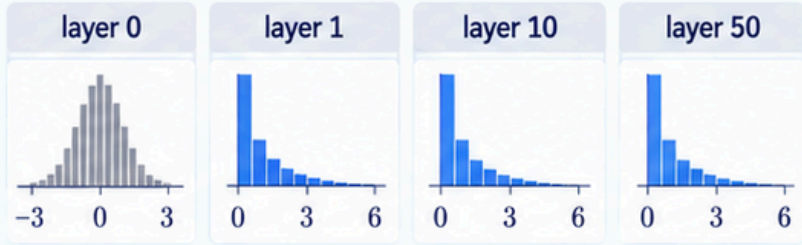


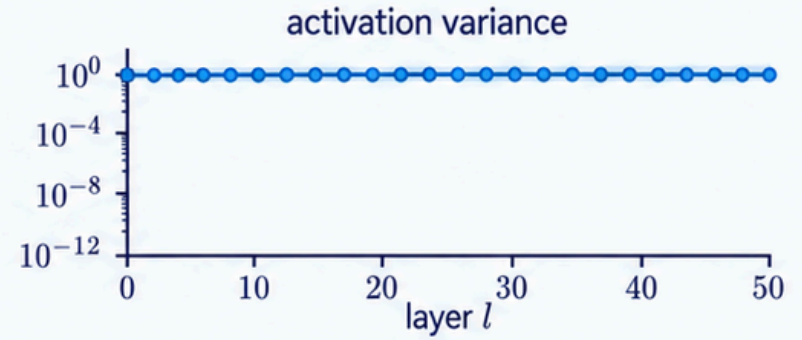


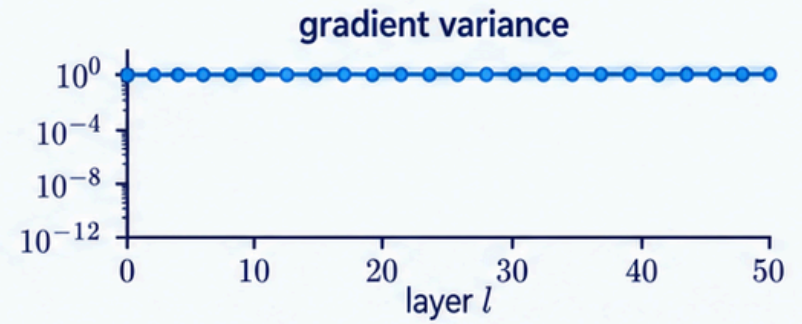


He (Kaiming) · ReLU

$W \sim \mathcal{N}(0, 2/n_{\text{in}})$





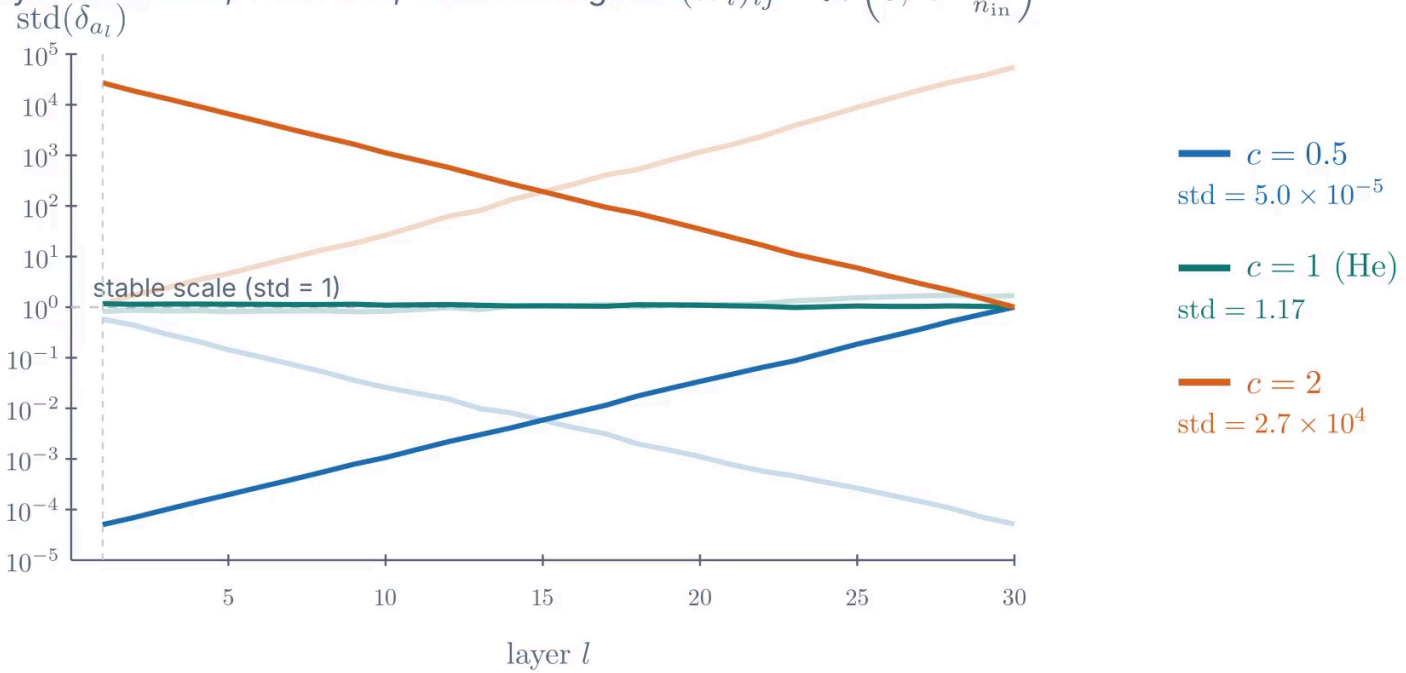


Match the initialization to the activation: Xavier for tanh, He for ReLU.

Signal propagation through depth

Backward: the same factor, from output to input

30-layer ReLU MLP, width 256, random weights: $(W_l)_{ij} \sim \mathcal{N}\left(0,\ c \cdot \frac{2}{n_{\text{in}}}\right)$



With He initialization both directions stay near 1: early layers can still learn.

Open animation

Part 3 · Optimization

1. Signal propagation — how scale moves through depth
2. Initialization — choosing the starting scale
3. **Optimization** — how gradients become steps
4. Normalization and regularization — stable scales and generalization
5. Residual learning and diagnostics — direct paths and debugging

> Part 2: choose Var(W) so that each layer keeps the variance; He for ReLU, Xavier for tanh.

SGD is simple but noisy

Mini-batch gradients estimate the dataset gradient:

$$J_{\mathcal{B}_t}(\theta) = \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} L_i(\theta), \qquad g_t = \nabla_\theta J_{\mathcal{B}_t}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta g_t$$

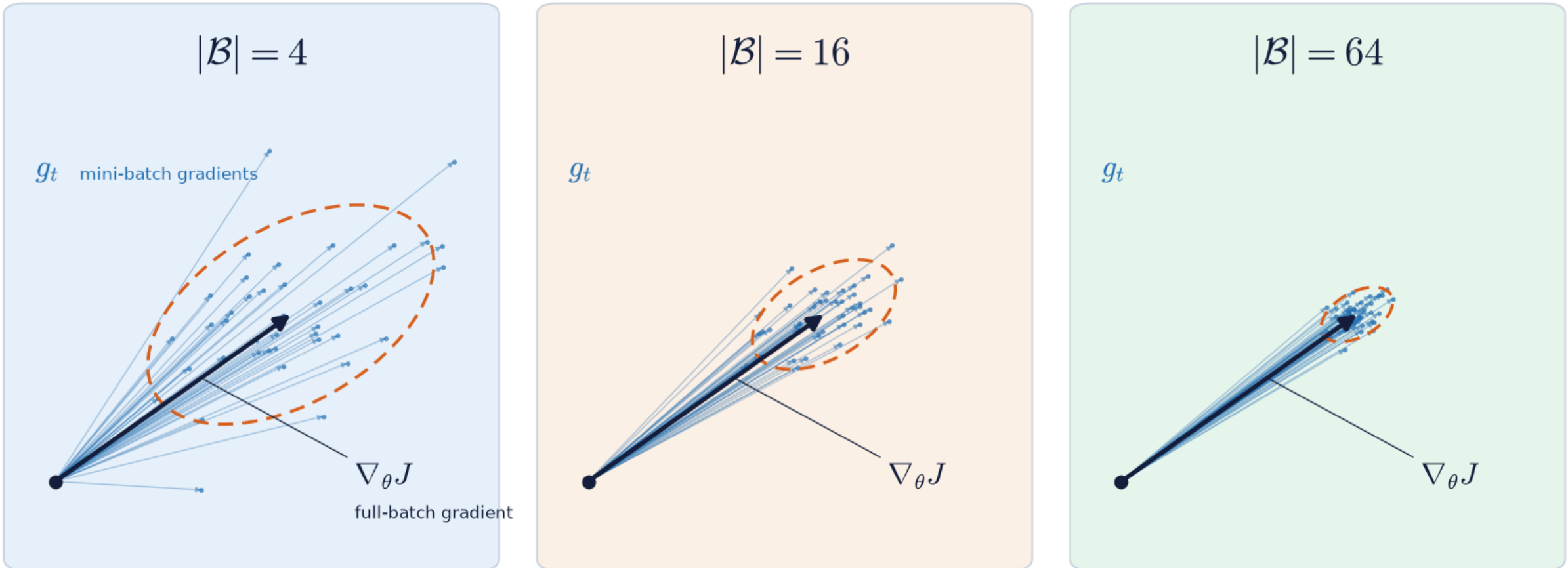
Noise can help exploration, but it can also slow progress.

Batch size controls the noise

With examples sampled independently, the mini-batch gradient is unbiased:

$$\mathbb{E}[g_t] = \nabla_\theta J(\theta_t), \qquad \mathrm{Cov}(g_t) \approx \frac{1}{|\mathcal{B}|}\mathrm{Cov}_i(\nabla_\theta L_i(\theta_t))$$

- the noise standard deviation falls like $1/\sqrt{|\mathcal{B}|}$: four times the batch, half the noise
- a larger batch costs more per step but allows a larger η (usually with warmup)
- returns diminish: past some size, more examples per step barely help



$\text{std} \propto 1/\sqrt{4} = 0.50$

$\text{std} \propto 1/\sqrt{16} = 0.25$

$\text{std} \propto 1/\sqrt{64} = 0.125$

dashed ellipse: 2 std of g_t around $\nabla_\theta J$ · simulated

Same expected direction, less noise: 4× the batch halves the standard deviation, at 4× the cost per step.

Momentum accumulates direction

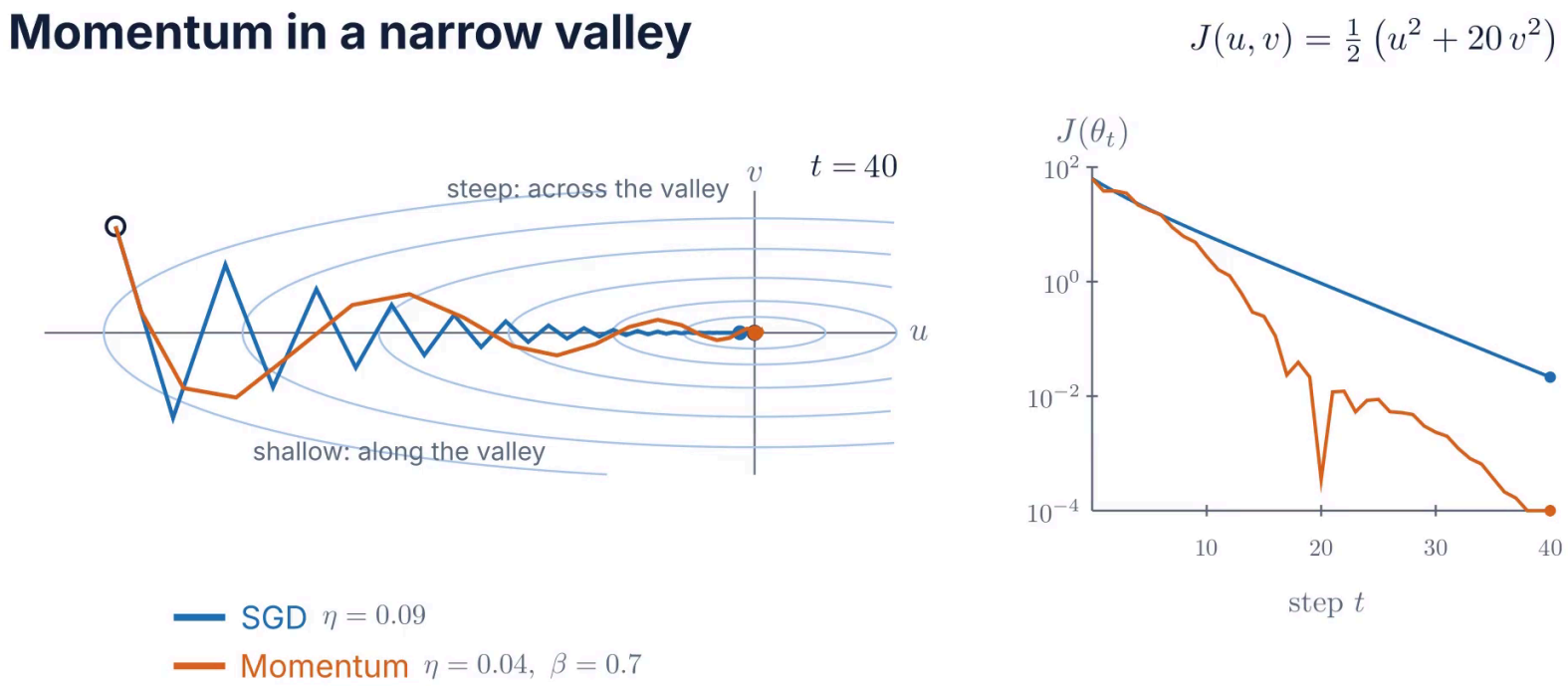
Momentum keeps a velocity vector:

$$v_t = \beta v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t$$

It can reduce oscillation and accelerate movement along persistent descent directions.

In a constant direction $v_t \to g/(1-\beta)$: the **effective step** is $\eta/(1-\beta)$, ten times larger for $\beta = 0.9$. Compare SGD and momentum at equal effective step.

Momentum in a narrow valley



After 20 steps: SGD J ≈ 0.93, Momentum J ≈ 0.0004. Momentum builds speed along the valley.

Open animation

RMSProp rescales each parameter

RMSProp keeps a running average of squared gradients:

$$s_t = \rho\, s_{t-1} + (1-\rho)\, g_t^2$$

$$\theta_{t+1} = \theta_t - \eta\, \frac{g_t}{\sqrt{s_t} + \epsilon}$$

Directions with consistently large gradients take smaller steps; flat directions take larger ones.

Operations on g_t and s_t are elementwise.

Adam combines both ideas

First moment (momentum) and second moment (RMSProp):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad s_t = \beta_2 s_{t-1} + (1-\beta_2) g_t^2$$

Bias correction and update:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1-\beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon}$$

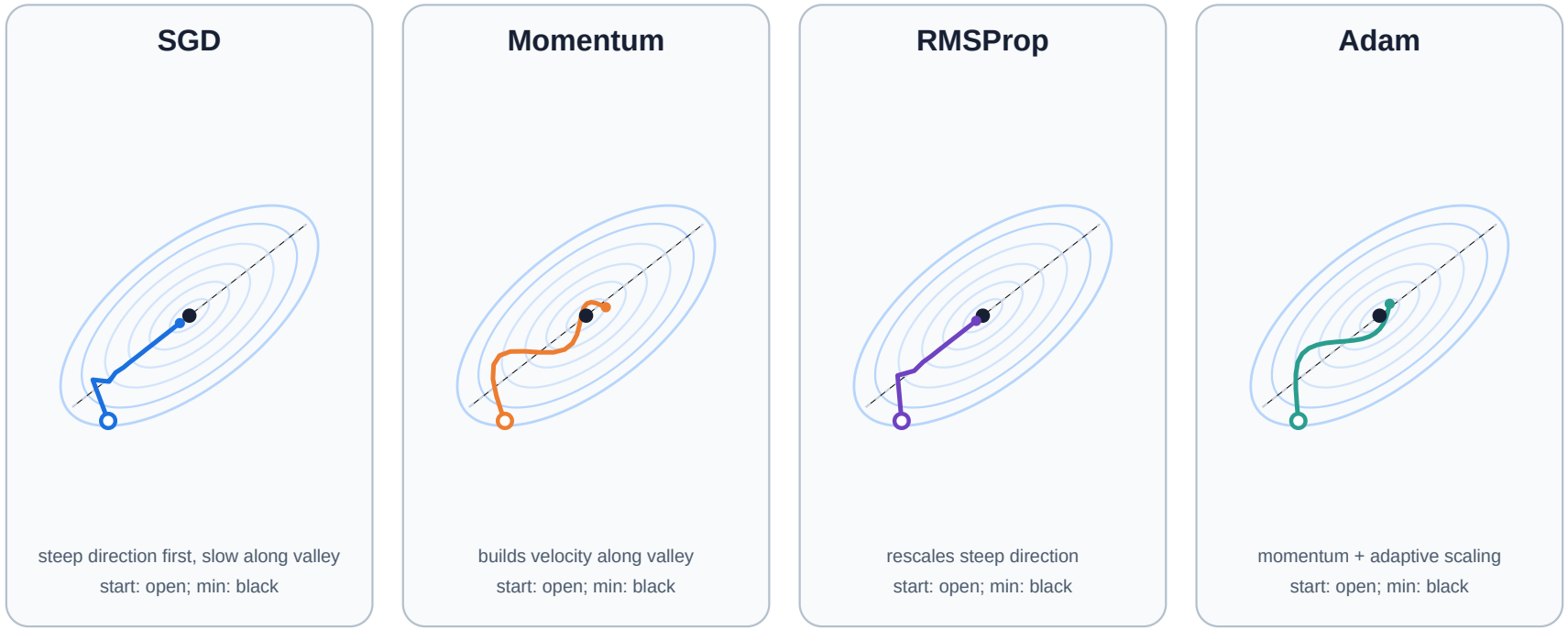
PyTorch defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

Comparing optimizers

Optimizer trajectories on an ill-conditioned quadratic

Same loss, same start and same minimum in every panel. The shallow direction runs along the long axis of the valley.

$$J(u,v) = 0.5\,(0.12\,u^2 + 1.5\,v^2)$$



The contrast appears because curvature is much larger across the valley than along it.

Learning rate schedules

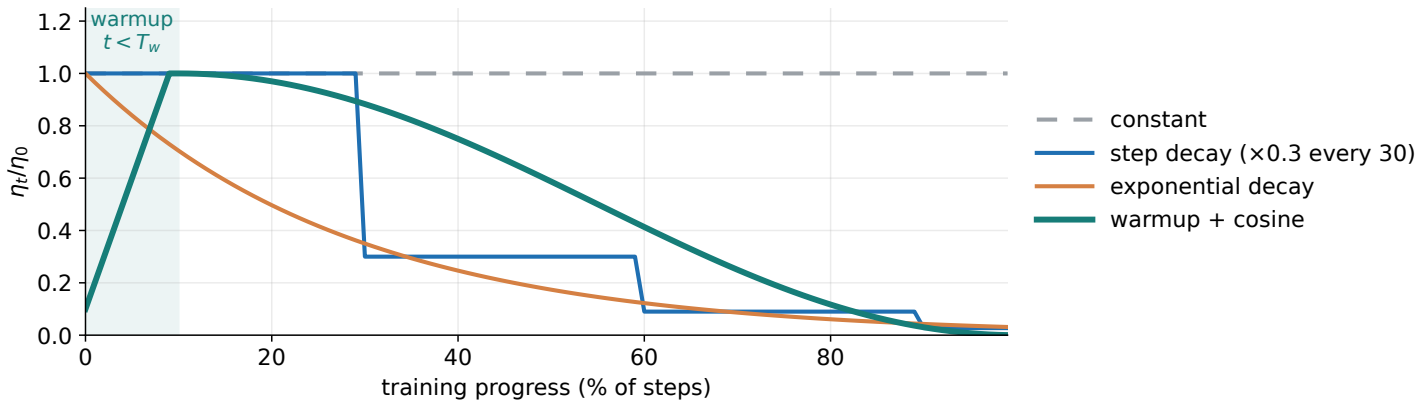
A fixed learning rate may be too large late in training or too small early in training.

Schedulers change η across time:

$$\eta_t = s(t)\,\eta_0$$

Common patterns: step decay, exponential decay, cosine annealing. Warmup + cosine over T steps:

$$s(t) = \frac{t}{T_w} \;\; (t < T_w), \qquad s(t) = \tfrac{1}{2}\left(1 + \cos\frac{\pi\,(t - T_w)}{T - T_w}\right) \;\; (t \geq T_w)$$



Gradient clipping

When a single batch produces a huge gradient, one update can undo many good ones.

Clipping by norm rescales the gradient before the step:

$$g_t \leftarrow g_t \cdot \min\left(1,\ \frac{c}{\|g_t\|}\right)$$

```
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

It treats the **symptom** of exploding gradients; initialization and normalization address the cause.

Part 4 · Normalization and regularization

1. Signal propagation — how scale moves through depth
2. Initialization — choosing the starting scale
3. Optimization — how gradients become steps
4. **Normalization and regularization** — stable scales and generalization
5. Residual learning and diagnostics — direct paths and debugging

> Part 3: the optimizer decides how gradients become steps: momentum, per-parameter scaling, schedules and clipping.

Normalization as a training tool

Normalization controls scale.

We already standardize inputs:

$$x' = \frac{x - \mathrm{mean}(x)}{\mathrm{std}(x)}$$

Batch Normalization extends this idea to internal activations.

Batch Normalization

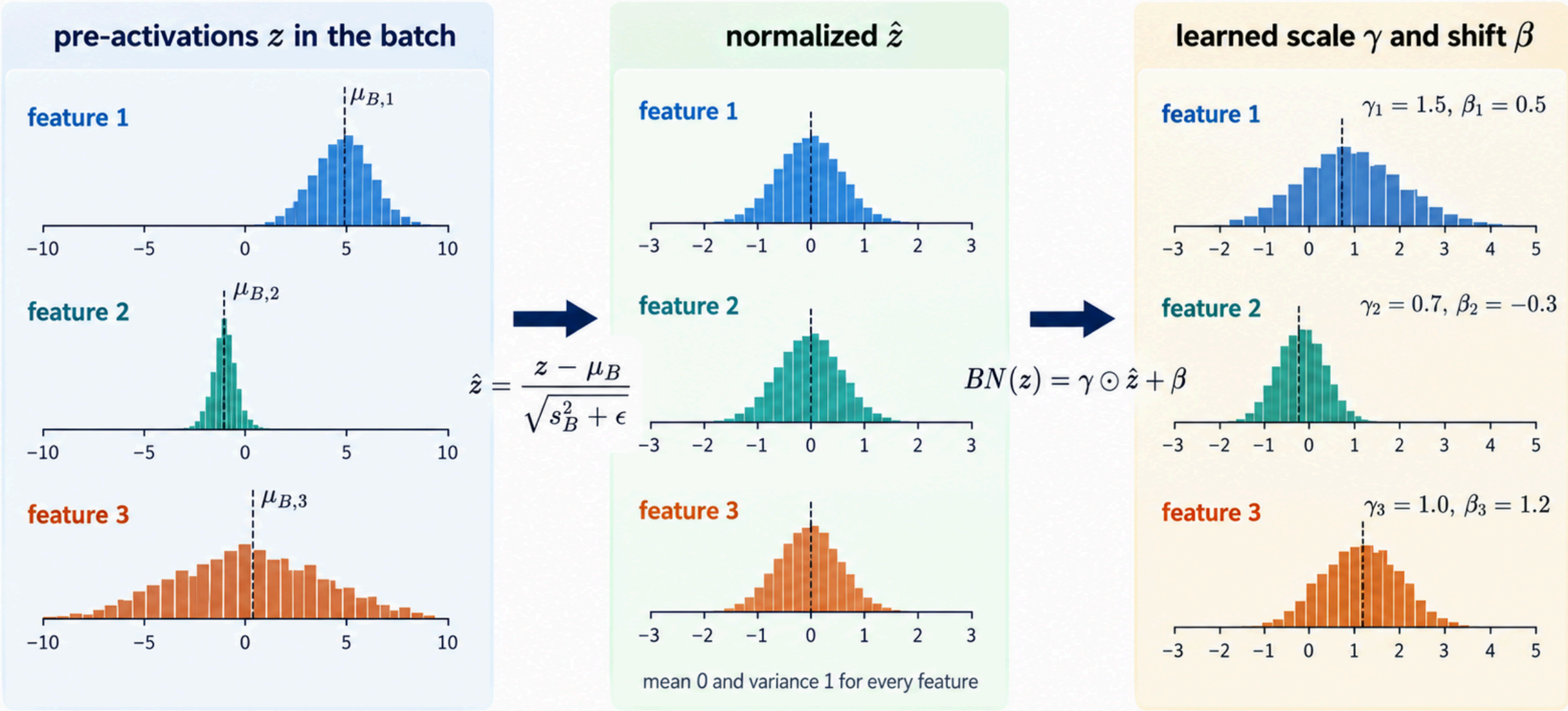
For pre-activations in a mini-batch $\mathcal{B}$:

$$\hat{z} = \frac{z - \mu_{\mathcal{B}}}{\sqrt{s_{\mathcal{B}}^2 + \epsilon}}$$

Then the layer learns a scale γ and a shift β:

$$\mathrm{BN}(z) = \gamma \odot \hat{z} + \beta$$

$\mu_{\mathcal{B}}$, $s_{\mathcal{B}}^2$: batch mean and variance. γ, β: BN parameters, not the optimizer's β.



Statistics are computed per feature, across the batch. The learned γ and β let the network recover any scale and offset it needs.

BatchNorm: training versus inference

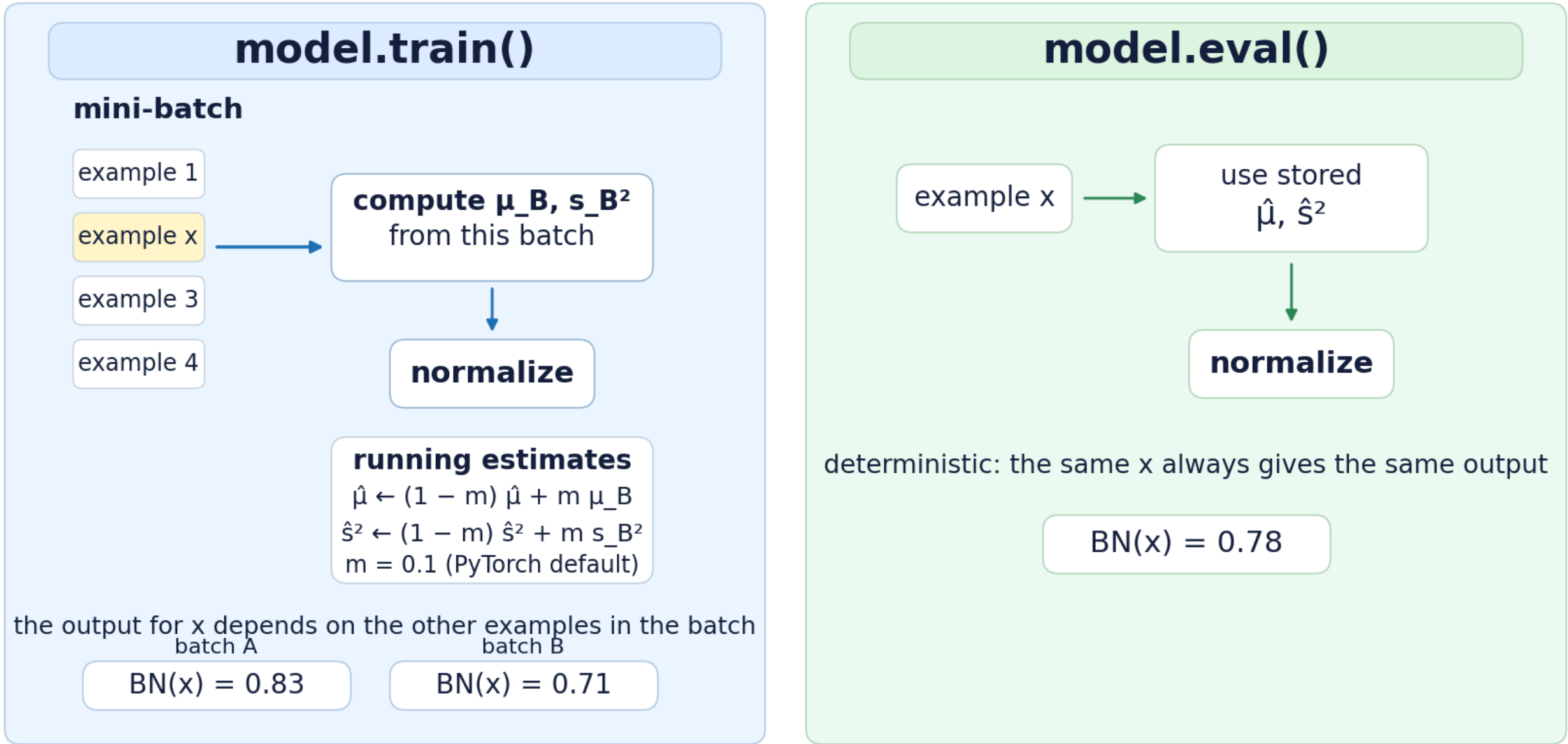
During training, $\mu_{\mathcal{B}}$ and $s^2_{\mathcal{B}}$ come from the current mini-batch.

During inference, the model uses running estimates accumulated during training:

$$\mu_{\text{run}} \leftarrow (1-m)\,\mu_{\text{run}} + m\,\mu_{\mathcal{B}}, \qquad m = 0.1 \ (\text{PyTorch momentum})$$

and the same for s^2. Typical placement: `Linear` → `BatchNorm1d` → ϕ.

```
model.train()   # batch statistics
model.eval()    # running statistics
```



Forgetting `model.eval()` at validation or test time is a common bug: predictions change with the batch.

Why does BatchNorm help?

The original motivation was to reduce "internal covariate shift" (Ioffe & Szegedy, 2015). Later experiments questioned that explanation (Santurkar et al., 2018).

A more accepted view:

- it keeps pre-activation scales stable across layers and updates;
- it makes the loss smoother, so larger learning rates stay stable;
- batch statistics add noise, which acts as a mild regularizer.

Costs: it depends on the batch size and behaves differently in training and inference.

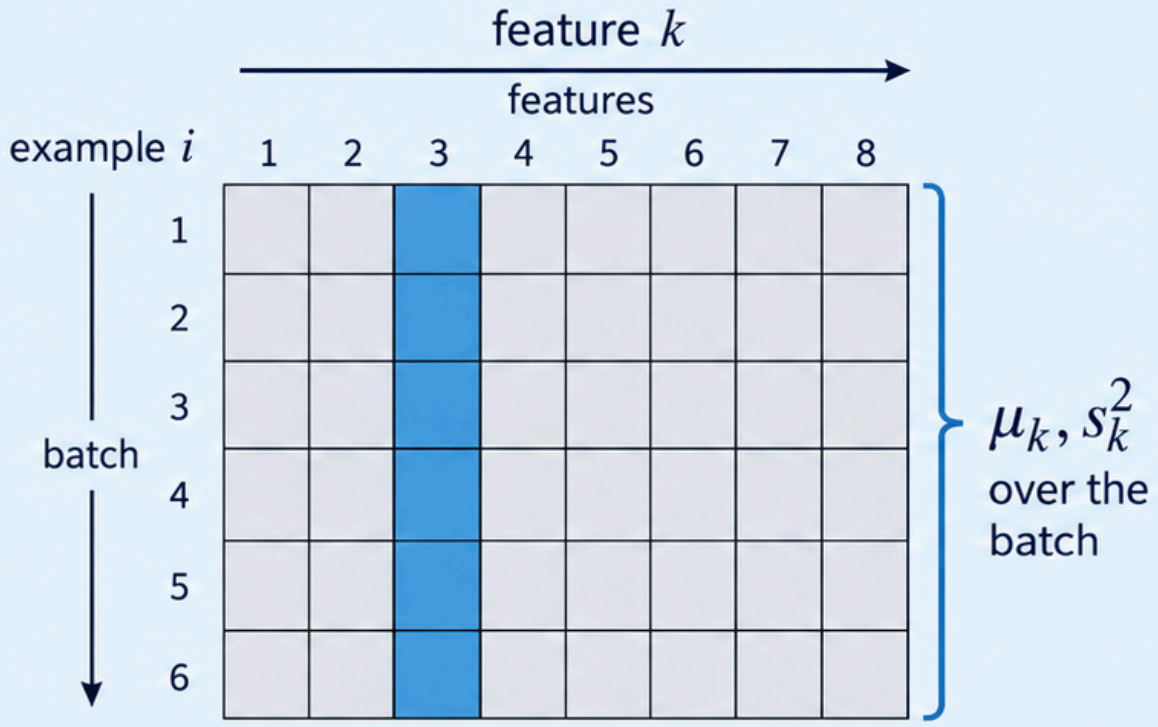
Layer Normalization

BatchNorm normalizes each feature **across the batch**. LayerNorm normalizes each example **across its features**:

$$\mu_i = \frac{1}{d}\sum_{k=1}^{d} z_{ik}, \qquad s_i^2 = \frac{1}{d}\sum_{k=1}^{d}(z_{ik}-\mu_i)^2, \qquad \mathrm{LN}(z_i) = \gamma \odot \frac{z_i - \mu_i}{\sqrt{s_i^2 + \epsilon}} + \beta$$

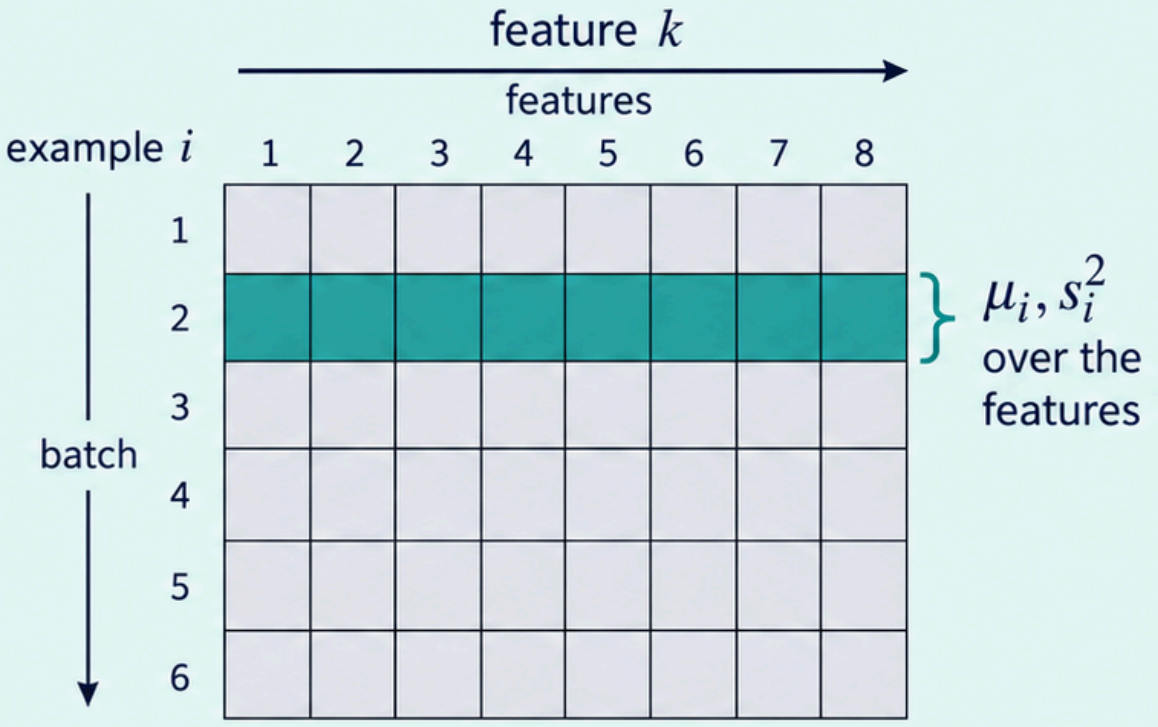
- the same computation in training and inference;
- works with batch size 1 and with sequences of different lengths;
- the standard choice in Transformers (S06).

BatchNorm



one mean and variance per **feature** • depends on the batch • running averages at inference

LayerNorm



one mean and variance per **example** • same in training and inference • works with batch size 1

Same formula $\gamma \odot \hat{z} + \beta$; BatchNorm averages down a column, LayerNorm across a row.

Capacity and the generalization gap

$$\text{gap} = J_{\text{val}}(\theta) - J_{\text{train}}(\theta)$$

- too little capacity: both objectives stay high (underfitting);
- enough capacity: both are low and the gap is small;
- large networks can drive $J_{\text{train}} \to 0$ even with random labels (Zhang et al., 2017): then the gap is what matters.

Regularization limits the **effective** capacity, the functions training actually reaches, without changing the architecture.

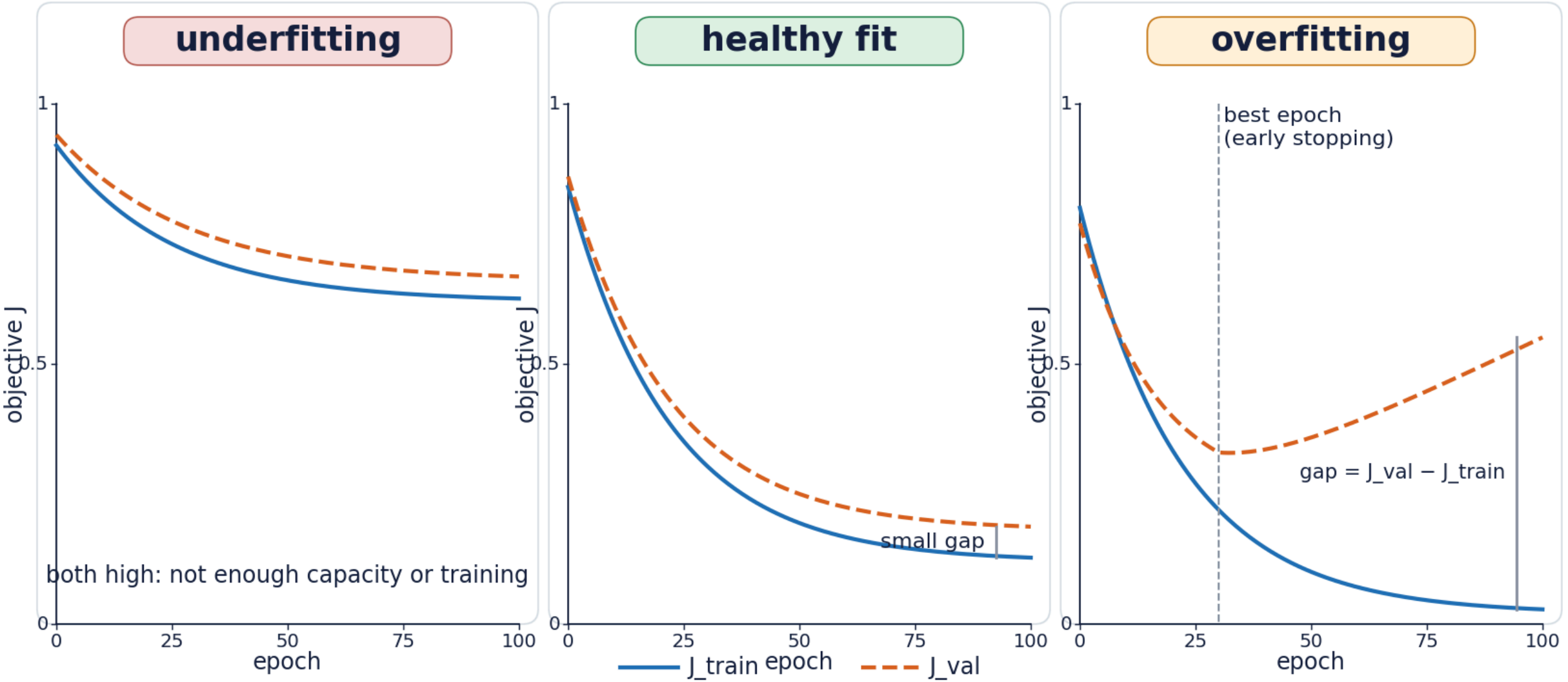
Regularization

Deep networks can fit complex functions.

Training loss alone is not enough.

We care about performance on unseen data:

$$J_{\text{train}}(\theta) \qquad J_{\text{val}}(\theta)$$



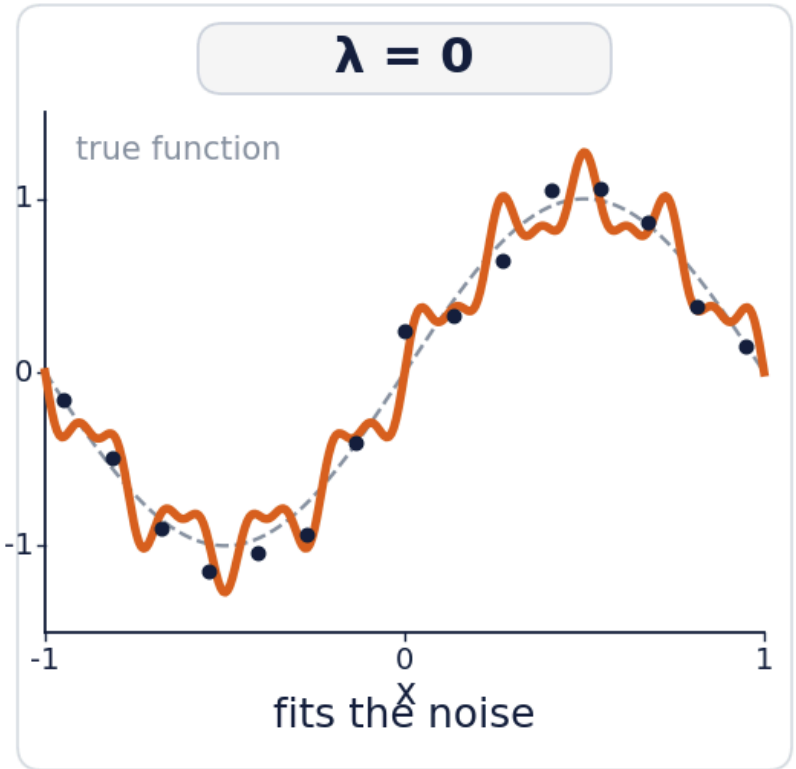
Read both curves together: the gap, not the training loss alone, tells you about generalization.

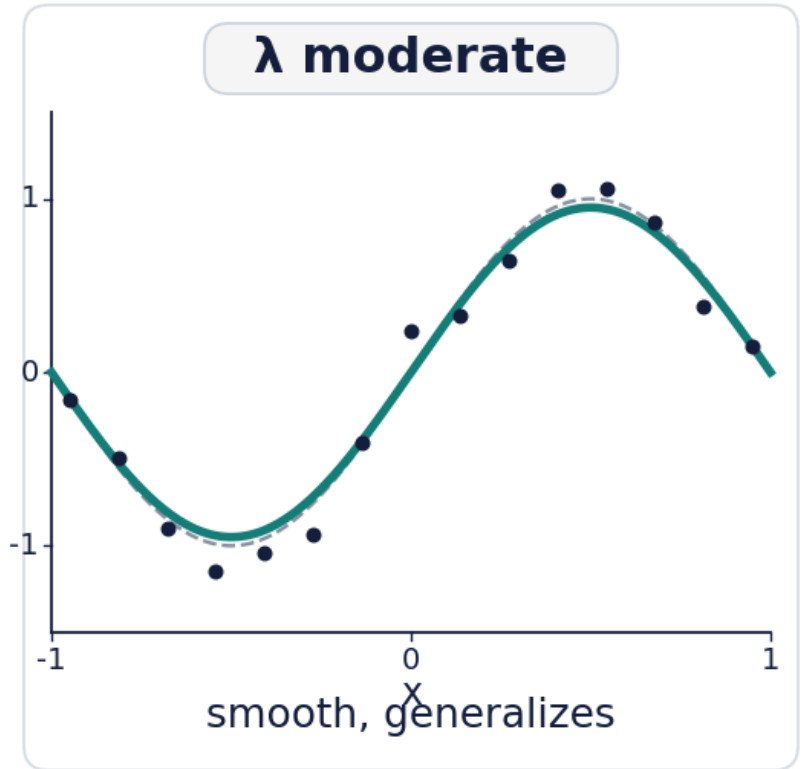
Weight decay

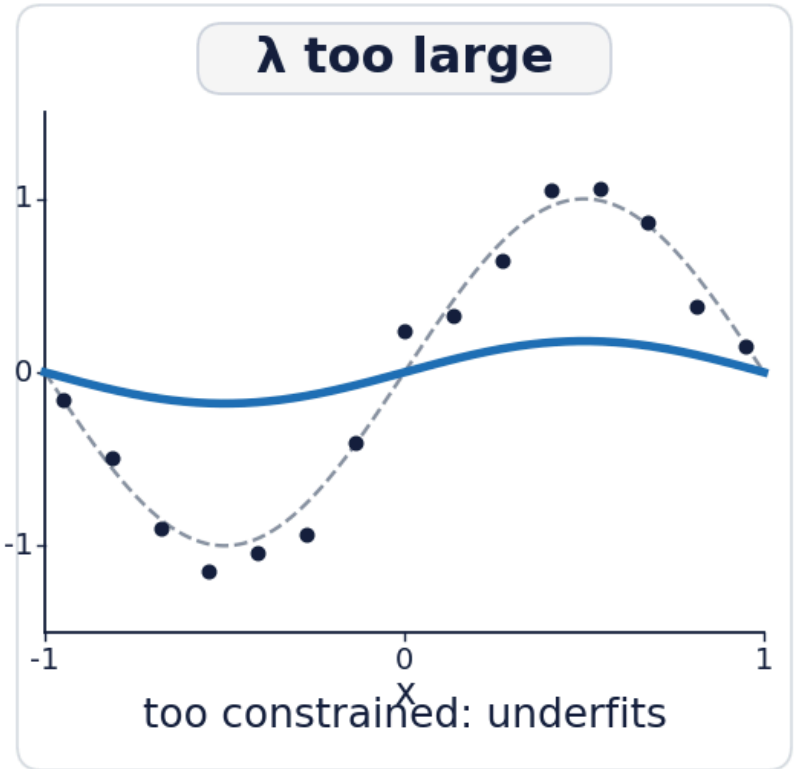
Weight decay discourages large weights:

$$J_\lambda(\theta) = J(\theta) + \lambda \|\theta\|_2^2$$

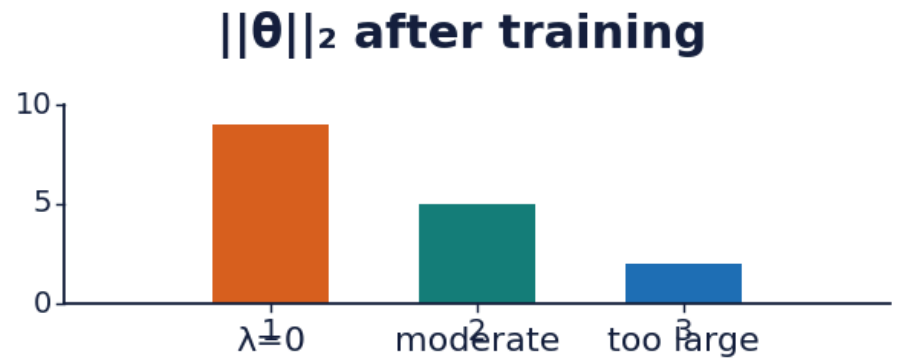
It changes the preference among solutions, not the architecture.







$$J_\lambda(\theta) = J(\theta) + \lambda \, ||\theta||_2^2$$



Weight decay prefers small weights; small weights give smoother functions.

Weight decay with Adam: AdamW

With SGD, adding $\lambda \left|\left|\theta\right|\right|_2^2$ to the loss and shrinking the weights at each step are equivalent (up to rescaling λ).

With Adam they are **not**: the penalty gradient is also rescaled by $\sqrt{\hat{s}_t}$.

AdamW decouples the decay from the adaptive step:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \epsilon} + \lambda \, \theta_t \right)$$

```
torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
```

Dropout

During training, each unit is dropped with probability p:

$$r_i \sim \text{Bernoulli}(1-p), \qquad \tilde{a} = \frac{r \odot a}{1-p}$$

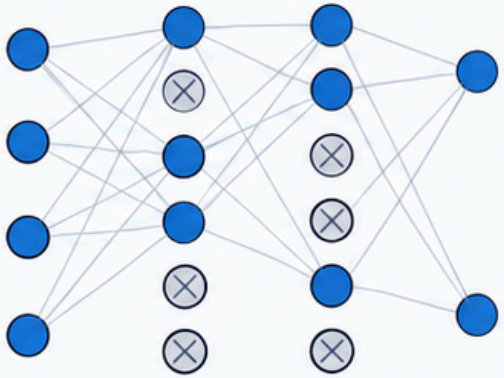
Dividing by $1-p$ keeps the expected activation unchanged, so at inference Dropout does nothing.

The network cannot rely on a single fixed path through hidden units.

`nn.Dropout(p)`: p is the probability of **dropping** a unit.

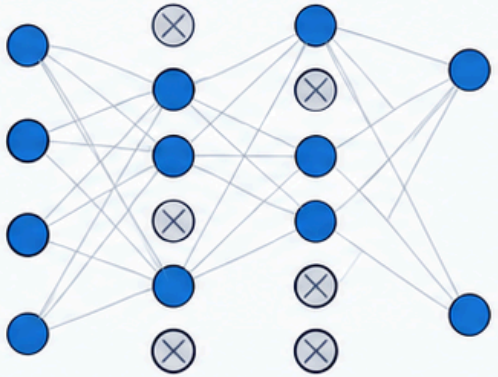
training: a new random mask r every step, p = 0.5

training step t



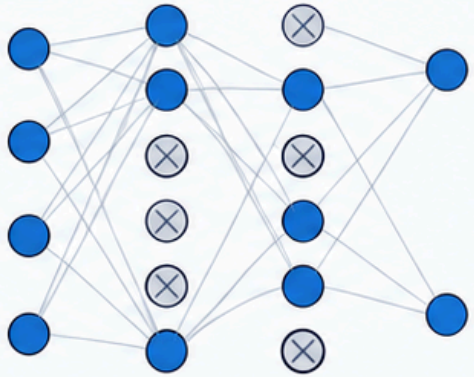
r = (1,0,1,1,0,0)

step t+1



r = (0,1,1,0,1,0)

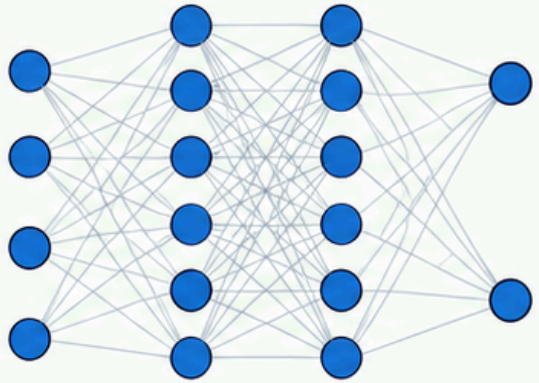
step t+2



r = (1,1,0,0,0,1)

$$\tilde{a} = (r \odot a) / (1 - p), \quad r_i \sim \text{Bernoulli}(1 - p)$$

inference (`model.eval()`)



no mask, no rescaling

Each step trains a different thinned network; at inference the full network approximates their average.
In `nn.Dropout(p)`, p is the probability of dropping a unit.

Early stopping

Keep the parameters with the best validation objective:

$$\theta^\star = \theta_{t^\star}, \qquad t^\star = \arg\min_t J_{\mathrm{val}}(\theta_t)$$

and stop when J_{val} has not improved for a fixed number of epochs (patience).

The number of training steps acts as a regularizer.

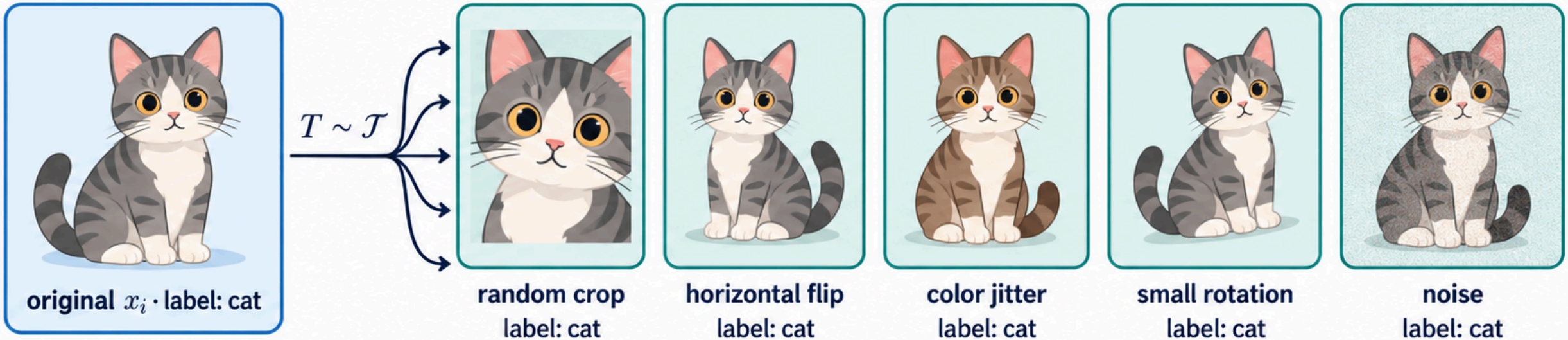
The S01 exercise already saved the best validation state this way.

Data augmentation

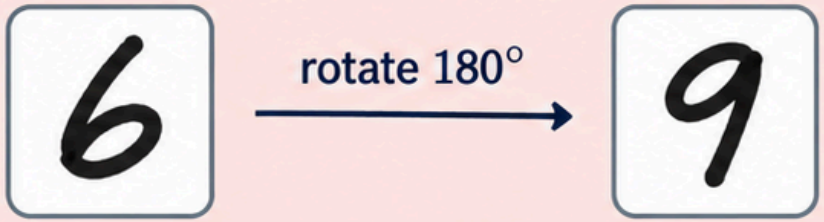
Train on transformed inputs that keep the label:

$$J_{\text{aug}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}_{T\sim\mathcal{T}}\Big[L\big(y_i,\ f_\theta(T(x_i))\big)\Big]$$

- images: crops, flips, color changes; signals: noise, shifts, scaling;
- T must not change the label: a rotated "6" may become a "9";
- it encodes invariances we know in advance. Central in S03 (vision) and S04 (self-supervised learning).



the transformation must not change the label



✗ **label changed: not a valid augmentation for digits**

Augmentation encodes invariances we know in advance: every $T(x_i)$ keeps the label y_i.

Part 5 · Residual learning and diagnostics

1. Signal propagation — how scale moves through depth
2. Initialization — choosing the starting scale
3. Optimization — how gradients become steps
4. Normalization and regularization — stable scales and generalization
5. **Residual learning and diagnostics** — direct paths and debugging

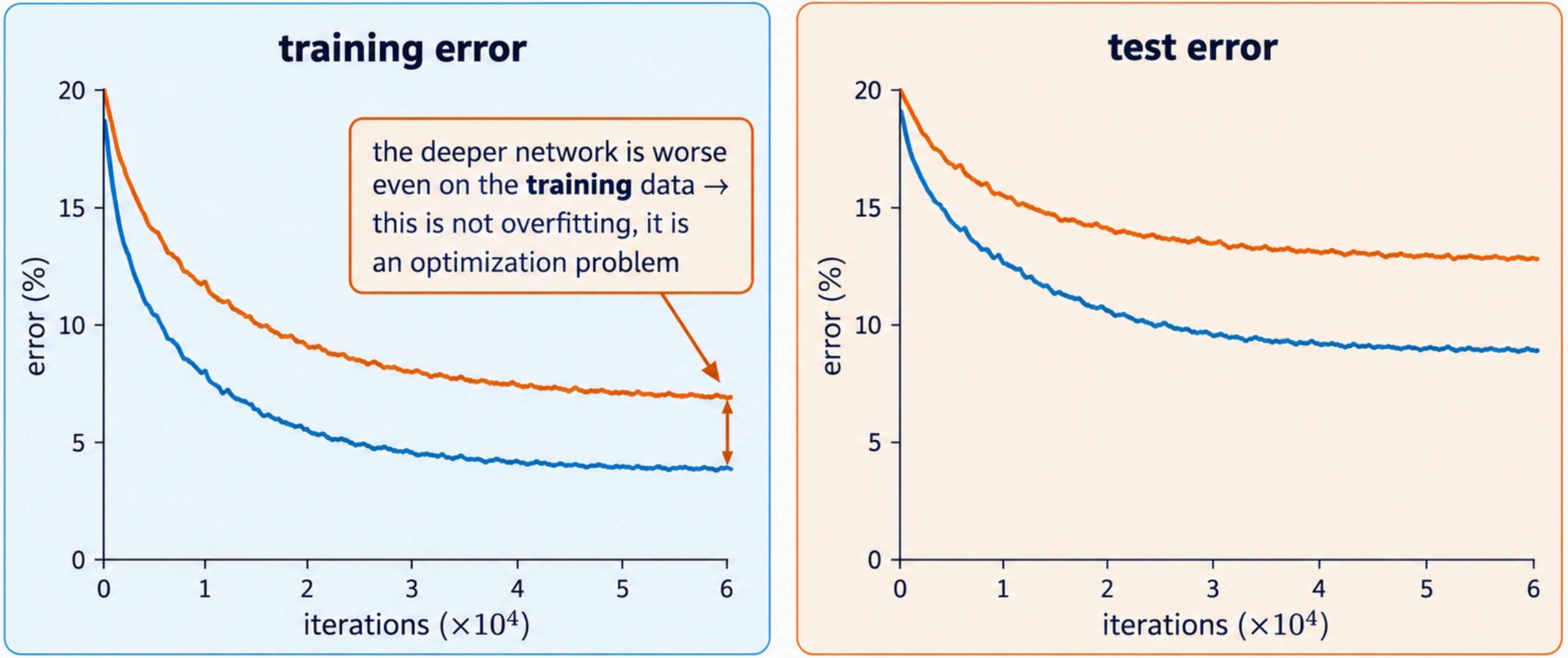
> Part 4: normalization keeps internal scales stable; regularization narrows the train/validation gap.

Deeper is not automatically better

A deeper plain network can represent at least as much as a shallower one: the extra layers could learn the identity.

Yet a 56-layer plain network reaches a **higher training error** than a 20-layer one (He et al., 2016).

Higher *training* error is not overfitting: it is an **optimization** problem.



Schematic after He et al., 2016 (CIFAR-10); values illustrative.

Deeper plain nets could learn identity layers, but **gradient descent does not find that solution.**

Residual learning

Instead of learning the whole transformation:

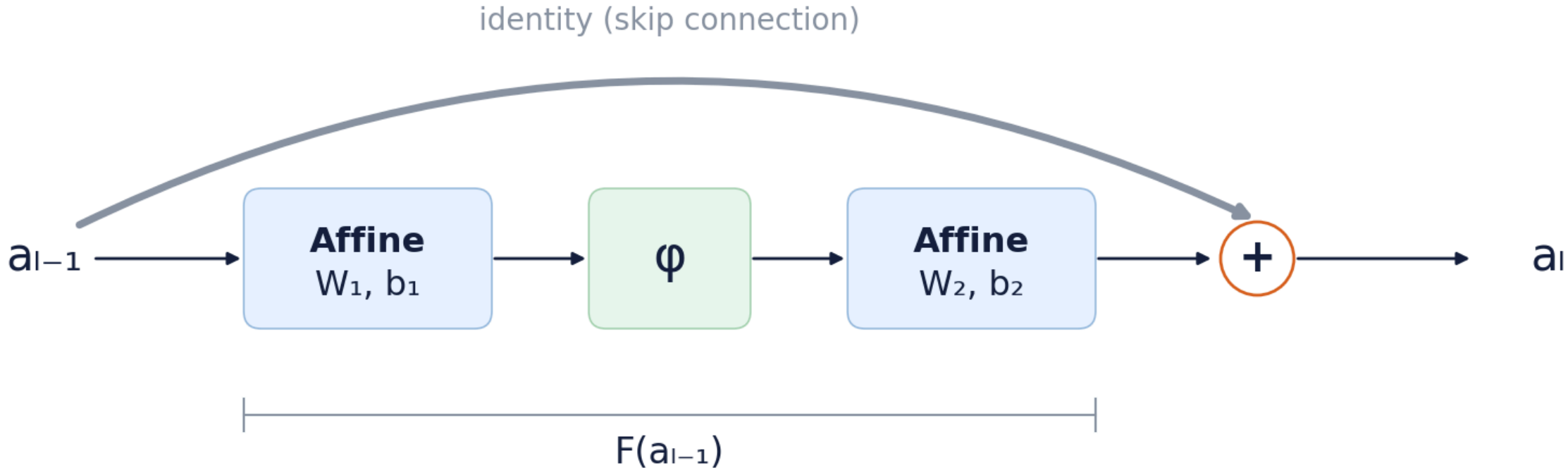
$$H(x)$$

learn a residual:

$$F(x) = H(x) - x$$

so that:

$$H(x) = F(x) + x$$



$$a_l = a_{l-1} + F(a_{l-1})$$

requires $F(a_{l-1})$ and a_{l-1} to have the same shape

a ResNet stacks many blocks

If the best block is close to the identity, F only has to learn a small correction.

A direct path for gradients

If:

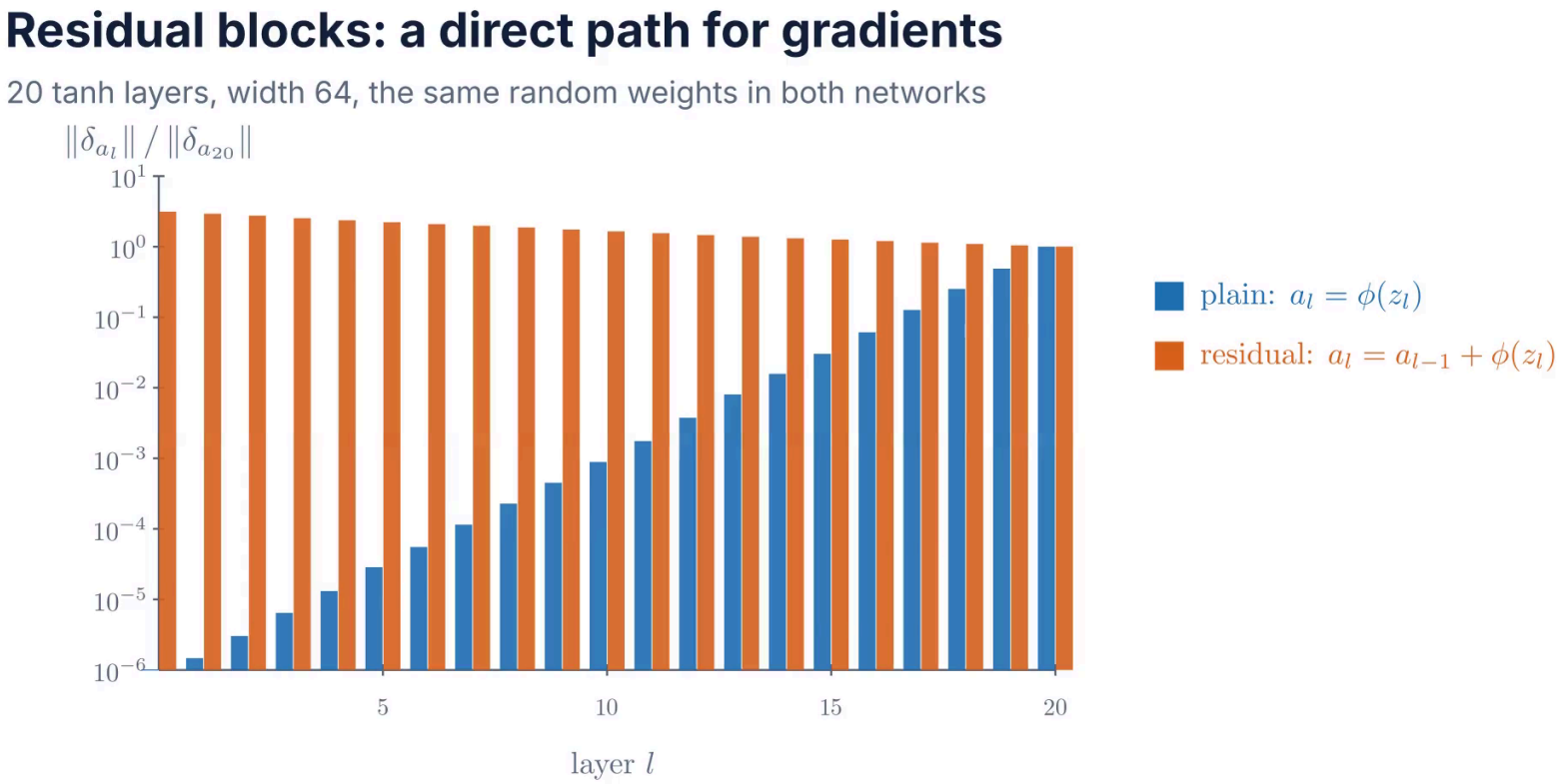
$$H(x) = F(x) + x$$

then the Jacobian is:

$$\frac{\partial H}{\partial x} = \frac{\partial F}{\partial x} + I$$

The identity path gives gradients a direct route through the block, even when $\partial F / \partial x$ is small.

Residual blocks: a direct path for gradients



Signal reaching layer 0, relative to the output: plain ≈ $6 \cdot 10^{-7}$, residual ≈ 3.1.

Open animation

A residual block in practice

F is a small network, for example two affine maps:

$$F(x) = W_2\,\phi(W_1 x + b_1) + b_2, \qquad H(x) = x + F(x)$$

- x and $F(x)$ are **added**, so they need the same shape; otherwise the shortcut uses a projection $W_s x$.
- The sums accumulate across blocks, so the scale of x can grow. **Pre-norm** normalizes the input of F:

$$H(x) = x + F\big(\mathrm{LN}(x)\big)$$

The identity path stays clean; this is the Transformer block of S06.

Diagnostics

When training fails, ask what failed.

Symptom	What to inspect
loss diverges	learning rate, gradient norms, clipping
loss does not move	initialization, dead activations, gradients
train improves but validation worsens	regularization, early stopping, data split
unstable curves	batch size, optimizer, normalization

Training diagnostics map

A training run should produce more than accuracy.

Track:

$$J_{\text{train}}, \quad J_{\text{val}}, \quad \|\nabla_{W_l} J\|, \quad \text{mean}(a_l), \quad \text{std}(a_l)$$

Before a long run: initial loss ≈ ln C · overfit one batch · finite non-zero gradients

symptom	what to measure	what to try
loss is NaN or diverges	gradient norms per layer; learning rate	lower η; warmup; gradient clipping
loss does not move	std(a_l) per layer; fraction of dead ReLUs; gradient norms	He initialization; Leaky ReLU / GELU; normalization; check η
train improves, validation worsens	gap $J_{val} - J_{train}$ over epochs	weight decay (AdamW); Dropout; data augmentation; early stopping
curves very noisy or unstable	batch size; BatchNorm mode	larger batch or smaller η; model.train() / model.eval(); LayerNorm
initial loss far from ln C	loss and label encoding; output layer	fix the bug before tuning anything

Diagnose with measurements, then change one thing at a time.

Sanity checks before training

Cheap tests that catch most bugs before a long run:

1. **Initial loss.** With C balanced classes and near-uniform predictions, cross-entropy starts near $\ln C$ (≈ 2.30 for $C = 10$).
2. **Overfit one batch.** A correct model and loop drive the loss on one small batch close to 0.
3. **Gradient norms.** Finite and non-zero in every layer after the first backward.
4. **Modes.** `model.train()` while training, `model.eval()` for validation (BatchNorm, Dropout).

From an experiment to a deployable model

A good curve is not enough: the run must be **comparable** and the model **usable** on new data.

Stage	Rule
Data	stratified train/val/test split; fit the scaler on **train only** (no leakage)
Tracking	one MLflow **run** = parameters + metrics per epoch + artifacts
Selection	choose with J_{val}; evaluate on test **once**
Artifact	weights **and** architecture, preprocessing statistics, input contract
Prediction	rebuild the model, `model.eval()`, same preprocessing as in training

Practice — Build a robust training recipe

Worked notebooks first, on Fashion-MNIST, one change per cell:

- 1 · Signal and initialization: per-layer std and gradients, Xavier vs He, initial loss vs ln C
- 2 · Optimization: η, momentum, Adam, batch size, warmup + cosine, clipping
- 3 · Normalization and regularization: weight decay, Dropout, early stopping, BatchNorm
- 4 · Residuals and diagnostics: residual blocks, pre-norm, overfit one batch

Then the integrating exercise · Forest Covertype (581k rows, 7 classes) · every run logged to **MLflow**: combine the pieces, explain the curves, deploy from the artifact.

Practice — Hyperparameter search

Notebook 5 · Optuna + MLflow · worked, on Fashion-MNIST

$$\lambda^\star = \arg\min_{\lambda} \; J_{\text{val}}\big(\theta^\star(\lambda)\big)$$

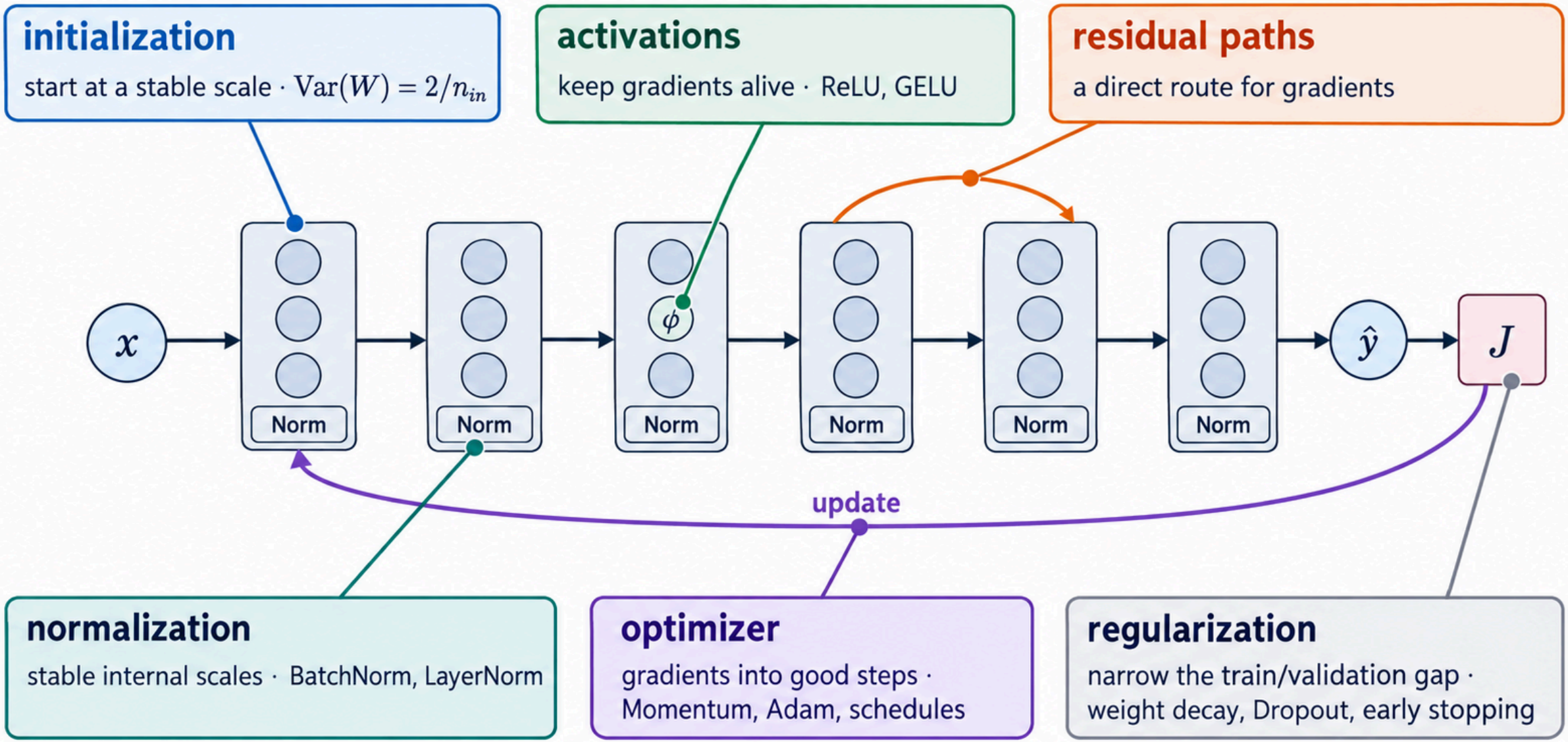
- search space: η and weight decay on a **log scale**, Dropout, depth, width, batch size, normalization;
- TPE proposes the next λ; a **median pruner** stops trials that already look worse;
- MLflow: one **parent** run per study, one **nested** run per trial; compare them with parallel coordinates.

> The best validation score of a search is optimistic: test once, after choosing.

Why deep training works

Deep networks train because several mechanisms cooperate:

$$\boxed{\text{initialization} + \text{activations} + \text{optimizer} + \text{normalization} + \text{regularization} + \text{residual paths}}$$



No single trick makes deep networks trainable: each mechanism fixes one way the signal or the optimization can fail.

Exit question

A 40-layer network has almost constant training loss.

What would you inspect first?

Choose two and justify:

- activation statistics;
- gradient norms;
- learning rate;
- initialization;
- BatchNorm mode;
- train/validation gap.